Advanced Machine Learning

Prof. Dr. Alexander Dück
alexander.dueck@dhbw.de

Distributed Semantics

Motivation

- Suchen Repräsentation für Text, die algorithmisch verarbeitet werden kann

Bag of Words, TF-IDF

Bag of Words

- Jedes Wort wird durch One-hot-vector repräsentiert
- Dimension = Größe des Vokabulars
- Bsp.:

Wort	Index	Vektor
awesome	0	$[1, 0, 0, 0, 0, 0]$
technology	1	$[0, 1, 0, 0, 0, 0]$
is	2	$[0, 0, 1, 0, 0, 0]$
hello	3	$[0, 0, 0, 1, 0, 0]$
world	4	$[0, 0, 0, 0, 1, 0]$
language	5	$[0, 0, 0, 0, 0, 1]$

input1: “Language technology is awesome”
Input2: “Hello Hello world”

Tokenization

input_{t1} = Language | technology | is | awesome
input_{t2} = Hello | Hello | world

Bag of words:

“technology” “language”

$$x_1 = [1, 1, 1, 0, 0, 1]$$

$$x_2 = [0, 0, 0, 2, 1, 0]$$

2x “Hello”

TF-IDF: Motivation

- Bag-of-Words zählt Wörter, aber:
  - häufige Wörter dominieren (z. B. und, der)
  - Nicht jedes häufige Wort ist informativ

- Wörter sind wichtig, wenn sie
  - im Dokument häufig
  - im Gesamtkorpus selten sind

→ gewichtete BoW-Repräsentation

TF-IDF: Formeln

- Für ein Wort w in Dokument d aus einem Korpus D:
- Term Frequency (TF):

$$\mathrm{tf}(w, d) = \frac{\#w \text{ in } d}{\#\text{Wörter in d}}$$

- Inverse Document Frequency (IDF):

$$\mathrm{idf}(w, D) = \log\left(\frac{|D|}{|\{d' \in D | w \in d'\}|}\right)$$

Anzahl an Dokumenten in D

Anzahl an Dokumenten in D, die w enthalten

- TF: lokale Wichtigkeit im Dokument
- IDF: globale Seltenheit im Korpus
- TF-IDF:

$$\mathrm{tfidf}(w, d, D) = \mathrm{tf}(w, d) \cdot \mathrm{idf}(w, D)$$

- Hoher Wert → charakteristisches Wort für dieses Dokument

TF-IDF: Verwendung

- Für jedes Dokument d lässt sich ein TF-IDF-Vektor $v_d \in \mathbb{R}^{|V|}$ erzeugen, der das Dokument repräsentiert ($|V|$: Größe des Vokabulars)
  - Ein Eintrag im TF-IDF-Vektor pro Wort im Vokabular

Mit diesen Vektoren kann man weiterarbeiten, z.B.

- Dokumente vergleichen (information retrieval):
  - Cosine Similarity: $\cos(v_d, v_q) = \frac{v_d \cdot v_q}{|v_d||v_q|}$ (d: Dokument, q: Query)
- Als Input für ML-Modelle nutzen (logistische Regression, SVM, …), z.B. für Textklassifikation

Bag of Words, TF-IDF

- **Reihenfolge der Wörter wird nicht berücksichtigt**, was für Aufgaben der natürlichen Sprachverarbeitung sehr wichtig sein kann.
- Nützlich bei der Dokumentenklassifizierung oder Information-retrieval, in Kombination mit der TF-IDF Gewichtung.
- BoW wurde häufig in „klassischen“ ML Methoden in Kombination mit Feature Engineering eingesetzt
  - Wort- und Character-N-grams (z.B. nicht gut → “nicht_gut”)
  - Part of Speech
  - Satzlänge
  - …

→ **Semantik wird nicht gelernt, sondern manuell als Features eingebaut.**

Word Embeddings & Distributed Semantics

Motivation I: Wozu Word Embeddings?

- Wörter müssen für Maschinen numerisch repräsentiert werden
- Klassische Repräsentationen (One-Hot):
  - sehr hochdimensional (30.000 – über 100.000 verschiedene Wörter)
  - keine Semantik (alle Wörter gleich weit voneinander entfernt)
- Sprache ist kontinuierlich, nicht diskret
- Ziel: ähnliche Wörter → ähnliche Vektoren

Motivation II: Distributionelle Semantik

- Bedeutung eines Wortes ergibt sich aus seinem Kontext
  - „You shall know a word by the company it keeps“
- Wörter mit ähnlicher Bedeutung treten in ähnlichen Kontexten auf
- Semantik als geometrische Struktur im Vektorraum
- Ermöglicht Vergleiche, Analogien und Generalisierung

Embeddings

- **Diskrete Variable** (z.B. Wort) soll in einen **kontinuierlichen** Wertebereich (Vektor in $\mathbb{R}^d$) übersetzt werden
- Idee und Methoden für Embeddings gibt es schon lange
- Viele Methoden
  - Word2Vec
  - FastText
  - GloVe
  - BERT, Elmo, …
- Unterschiedliche Ebenen für Embeddings
  - Character, Word, Paragraph, Dokument

Embeddings

- **Idee:** Bedeutung eines Wortes ergibt sich aus seinem Kontext, also aus Wörtern, die häufig in der Nähe vorkommen
  - „You shall know a word by the company it keeps“ (Firth, J. R. 1957:11)
- **Eigenschaften:** Wörter mit ähnlicher Bedeutung sollen ähnliche Vektoren haben
  - Tiger und Löwe sind sich ähnlich, weil sie häufig mit ähnlichen Wörtern genannt werden (z.B. Dschungel, Raubkatze, …)

Distributed Semantics – Intuition

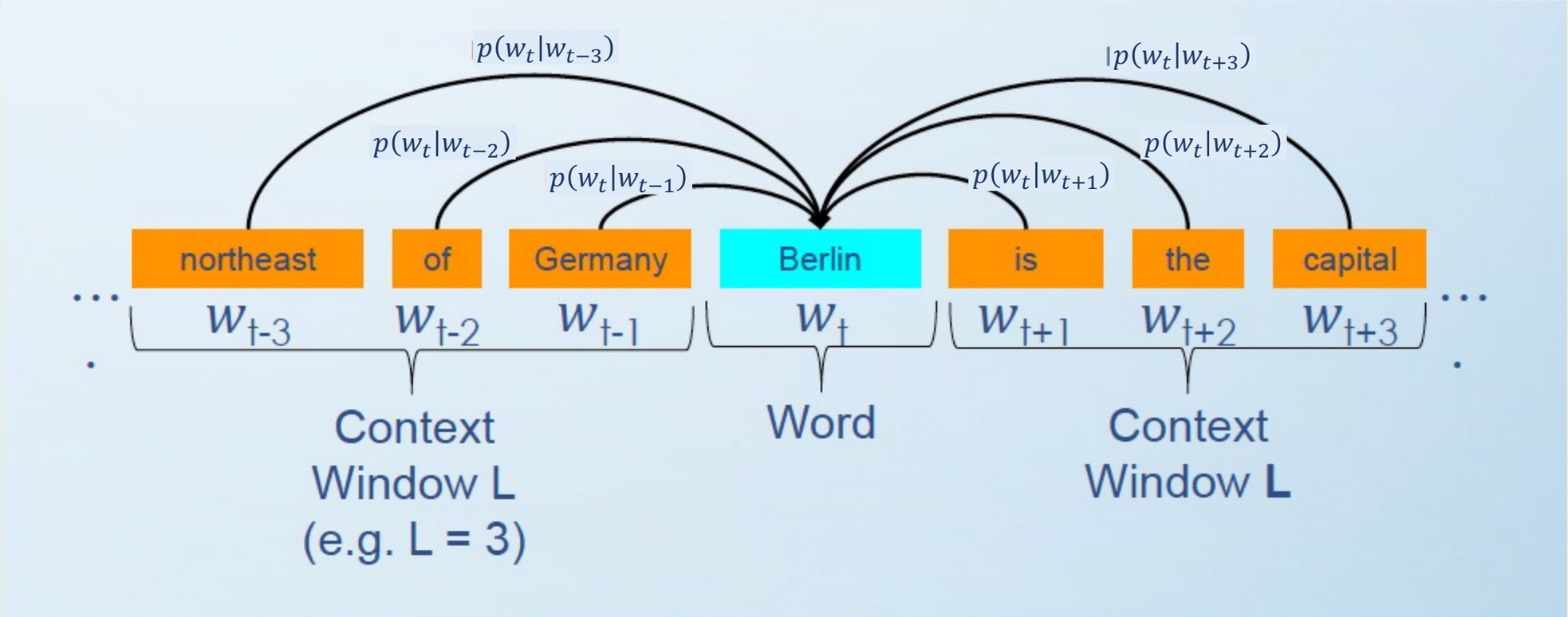
- Tun wir so, als würden wir das Wort ***ongchoi*** nicht kennen
- Wir beobachten das Wort im folgenden Kontext:
  - “Ongchoi schmeckt am besten mit Reis”
  - “Anschließend Ongchoi mit Knoblauch sautieren”
- Viele Wörter, die mit Ongchoi auftreten(Knoblauch, Reis, sautieren, schmeckt), kennen wir aus dem Bereich **Kochen**

Word2Vec

- Zwei Varianten: *CBOW* (continous bag-of-words) und *skip gram*
- Trainingsdaten: Großer Textkorpus (z.B. Wikipedia, Common Crawl, Twitter…)
- Jedes Wort aus festem Vokabular wird durch einen Vektor repräsentiert
- Für jede Position i im Text betrachten wir
  - Das zentrale Wort w_i
  - Die Kontextwörter c $(w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$
- Trainieren ein Modell zur Vorhersage von w_i aus c (CBOW) oder umgekehrt (skip gram) anhand der Wortvektoren
- Wortvektoren werden angepasst um die Vorhersage von Wahrscheinlichkeiten für die tatsächlichen Wörter zu maximieren

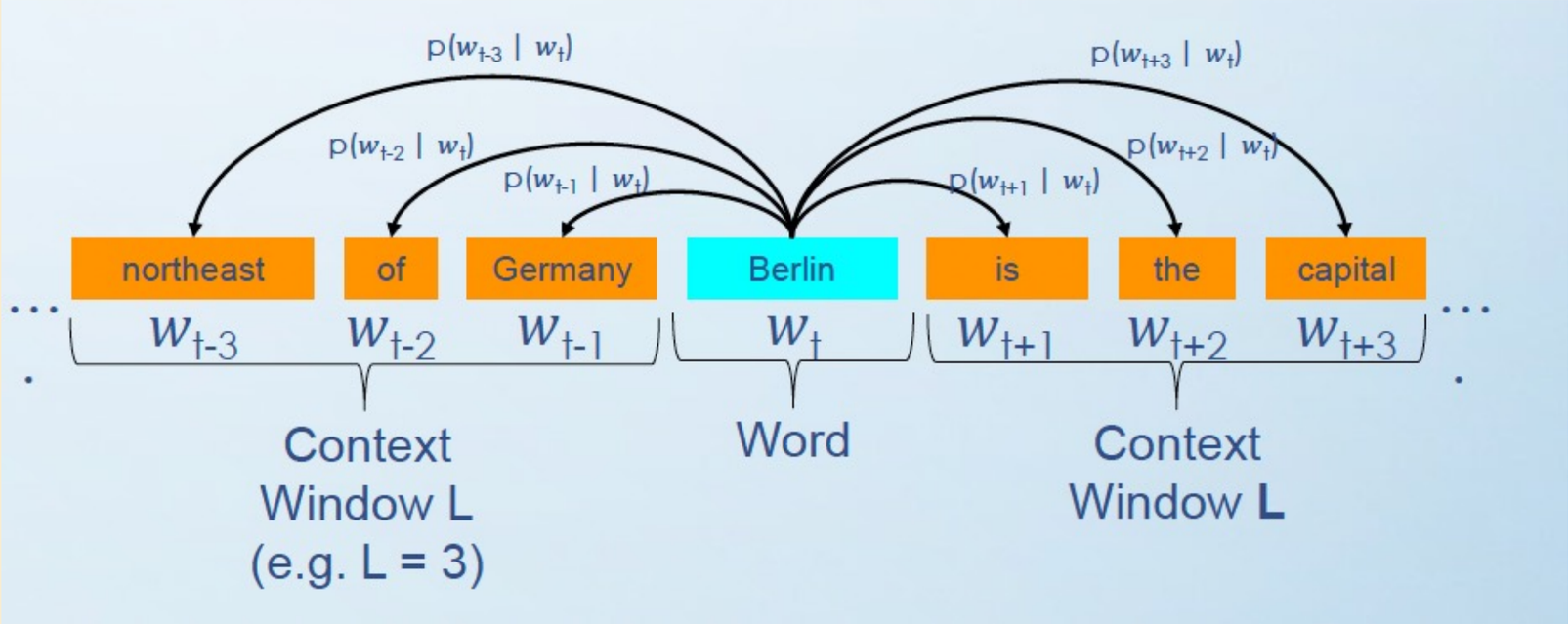
Word2Vec – CBOW

- **CBOW:** Sage zentrales Wort (hier: “Berlin”) basierend auf Kontextwörtern vorher
- Vorhersage basierend auf Mittelwert der Kontextwörter



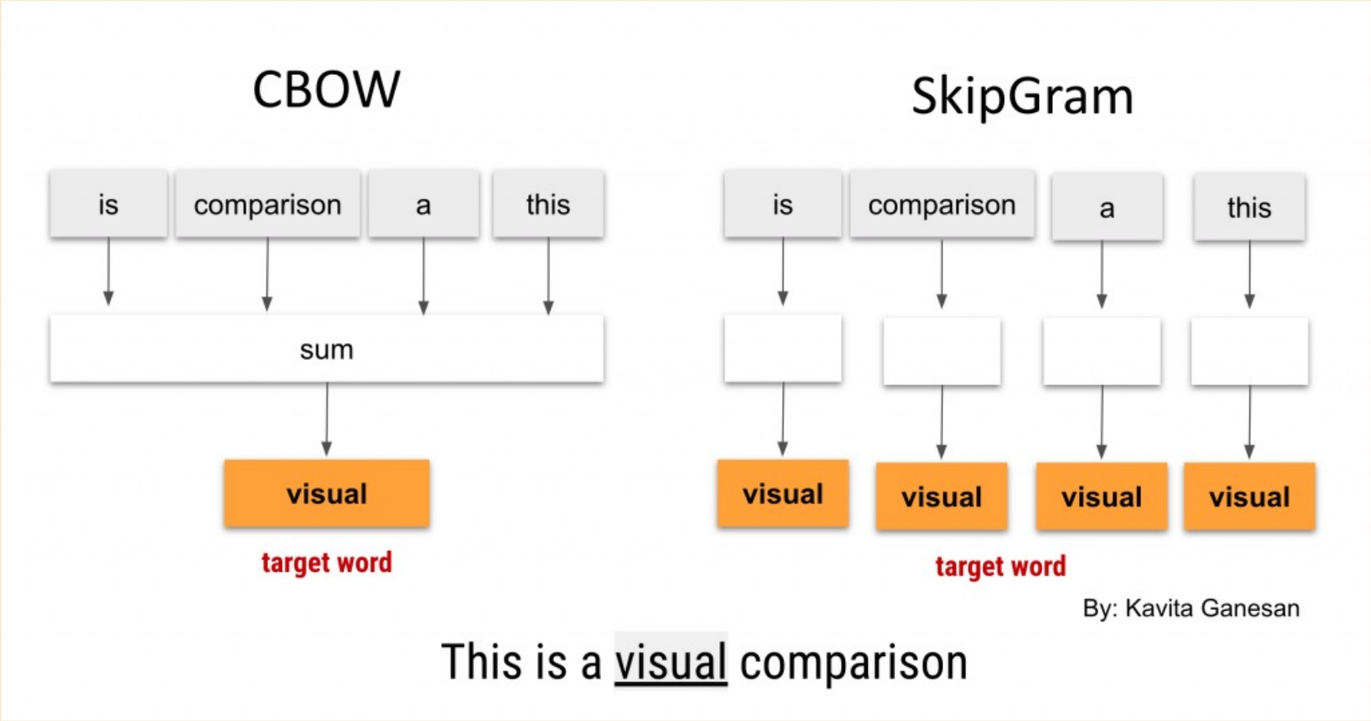
Word2Vec – Skip Gram

- **Skip Gram:** Sage die Kontextwörter basierend auf zentralem Wort vorher (Alternative Sichtweise: Sage das zentrale Wort basierend auf einem einzelnen Kontextwort vorher)



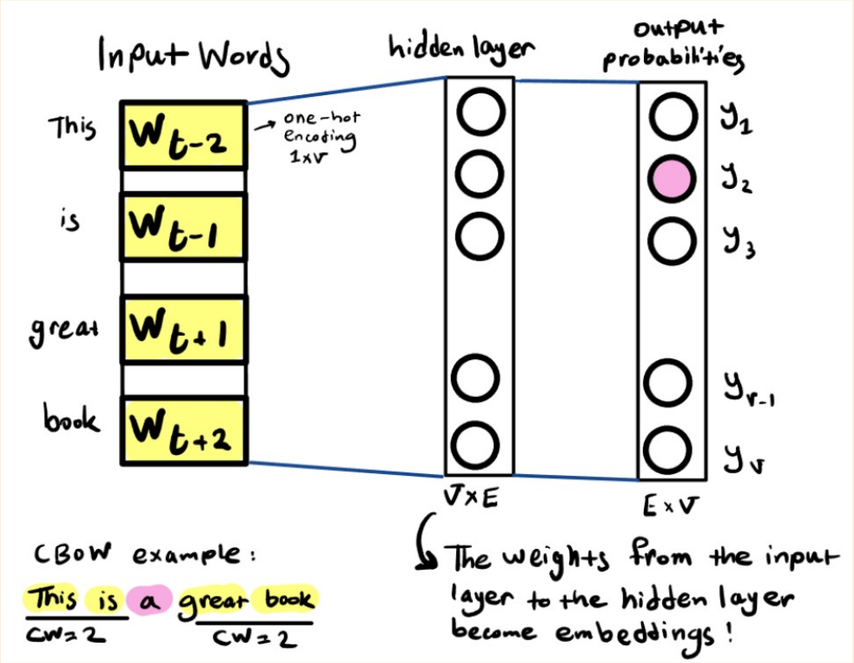
Word2Vec – CBOW vs Skip Gram

- CBOW: Sage zentrales Wort (hier: “visual”) basierend auf Kontextwörtern vorher
- Skip Gram: Sage die Kontextwörter basierend auf zentralem Wort vorher (Alternative Sichtweise: Sage das zentrale Wort basierend auf einem einzelnen Kontextwort vorher)
- Das Training erfolgt in jedem Fall mit einem MLP mit einem hidden layer
- Die Parameter des MLP sind die gesuchten Embeddings



Word2Vec – Embeddings

- Technisch: Wörter w_i sind one-hot-encoded
- Input des Netzwerks ist bei
  - CBOW Mittelung über alle Kontextwörter: $x = \frac{1}{L}\sum_{i=1}^{L} w_i$
  - Skip Gram das zentrale Wort w_i (one-hot vector)
- $M \in \mathbb{R}^{d \times V}$ ist die Parametermatrix des hidden layers
- V: Vokabulargröße, d: Embedding-Dimension
- $h = Mx$ → wählt Spalte i (Skip Gram) oder eine Mittelung der Spalten von M zu den Kontextwörtern (CBOW) aus
- → jede Spalte von M entspricht dem Word-Embedding eines Wortes
- Output des Netzwerks: $y = M'h \ \left(M \in \mathbb{R}^{V \times f}\right)$
- → die Zeilen von M' können ebenfalls als Embeddings dienen



https://huggingface.co/spaces/hesamation/primer-llm-embedding?section=word2vec

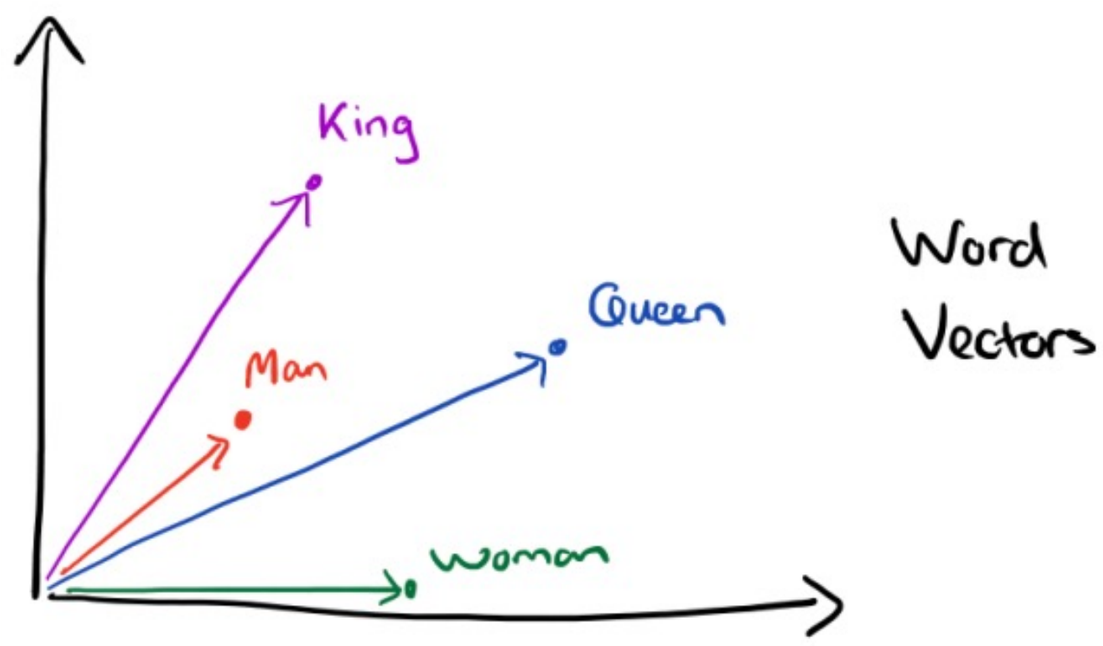
Hinweis: Es gibt Darstellungen, bei denen die Zeilen von M den Embeddingvektoren entsprechen; das hängt von der Konvention ab.

Word2Vec Parameter

- Größe des Kontextfensters
- Minimale Termhäufigkeit
- Dimension der Embeddings
- CBOW vs Skip Gram
- Anzahl an Epochen

Embeddings erfassen Beziehungen zwischen Wörtern

- Vektoren eigenen sich zur Beantwortung von Analogiefragen der Form A ist zu B wie C zu ???



→ Siehe Übung

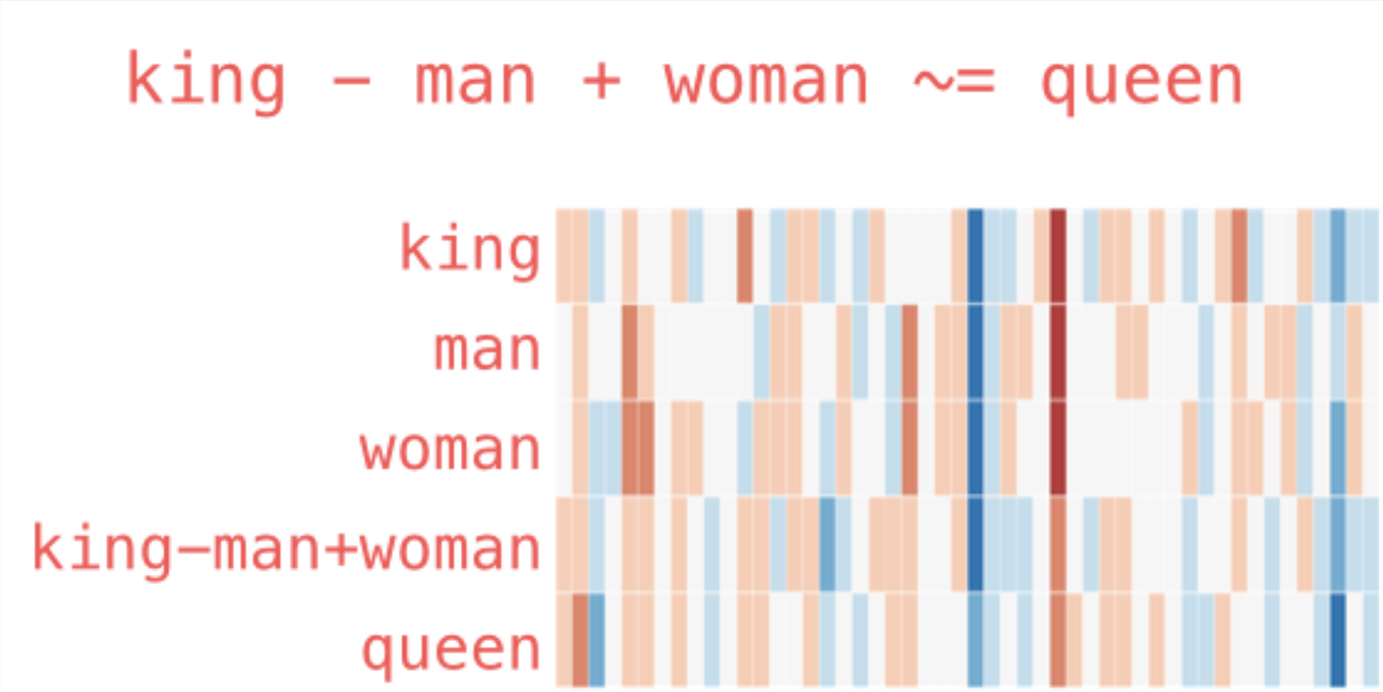
https://blog.acolyer.org/2016/04/21/the-amazing-power-of-word-vectors/

Bias in Embeddings

- Word-Embeddings verstärken Bias
  - Bekanntestes Beispiel **Man is to Doctor as Woman is to** …
- Caliskan et al. (2017) haben 200 Sentiment Analyse Systeme untersucht und festgestellt, dass **afroamerikanische Vornamen** zu einem **negativen Sentiment**führen.
- Analysen im Bereich hatespeechdetectionhaben ergeben, dass **nur die Nennung von Minderheiten** (Blind, Frau, Homosexuell, …) zu einer **Klassifikation als Hate Speech** führen.

Aylin Caliskan, Joanna J. Bryson, & Arvind Narayanan (2016). Semantics derived automatically from language corpora necessarily contain human biases. CoRR, abs/1608.07187.

Visualisierung von Embeddings



https://jalammar.github.io/illustrated-word2vec/

gensim: Word Embedding Bibliothek

- Enthält Vielzahl von Embedding-Methoden
  - Word2Vec
  - Doc2Vec
  - FastText
  - …
- Lookup, Ähnlichkeitsberechnung, …
- Verschiedene vortrainierte Embeddings lassen sich herunterladen

Subword-based Word Vectors: Motivation

Wortverteilung aus deutscher Wikipedia:

- 2.328.761 Artikel
- **10.116.714 Token** mit 687.524.490 Nennungen
- Token mit Häufigkeit < 5: 7.959.628 (78.6%)
- Token mit Häufigkeit < 1: 5.640.211 (55.75%)

→ Die meisten Token sind sehr selten! → Kriegen kein oder kein gutes Embedding dafür!

FastText Embeddings

- Erweiterung von Word2Vec
- Wörter werden als char-n-grams und dem Wort repräsentiert
  - **where** = <wh, whe, her, ere, re>, <where>
- Embedding berechnet sich aus der Summe der einzelnen Repräsentationen (N-Gramm und Wort)
- N-Grams der Länge 3-6

→ Generiert bessere Embeddings für seltene Wörter

→ Ermöglicht es mit Out-of-vocabulary umzugehen

Enriching Word Vectors with Subword Information, Bojanowski, Grave, Joulin and Mikolov. FAIR. 2016. https://arxiv.org/abs/1607.04606

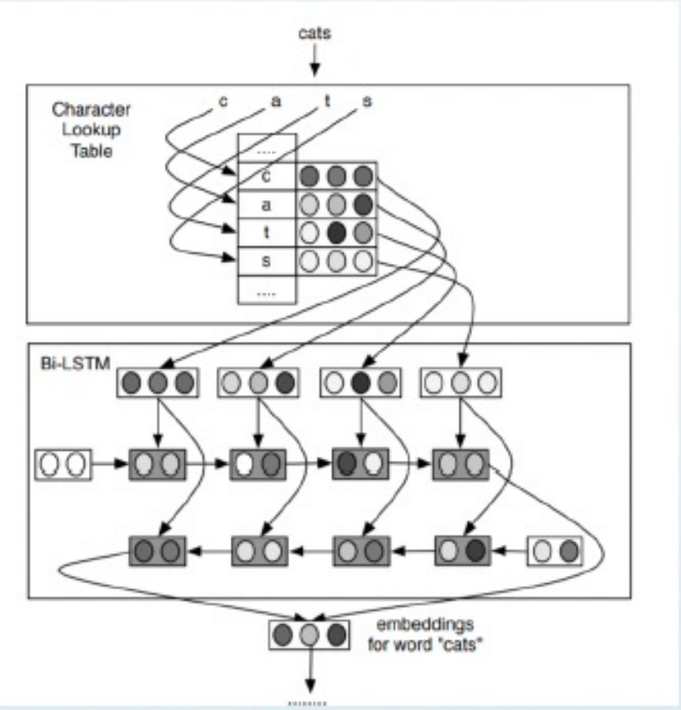
Character Embedding

- Motivation:
- Word Embeddings:
  - Out-of-vocabulary Problem
  - Keine Morphologie (verschiedene Embeddings für Katze und Katzen)
- FastText: feste n-Gramme
- Keine Embeddings für seltene Wörter und Rechtschreibfehler
- Character Embeddings
  - Vollständig OOV-fähig
  - Keine Tokenisierung notwendig
  - Robust gegenüber Rechtschreibfehlern

Ling, W., Luís, et al. (2015). Finding Function in Form: Compositional Character Models for Open Vocabulary Word Representation. EMNLP.
Zhang X., et al. (2015). Character-level convolutional networks for text classification. In Proceedings NIPS

Character Embedding

- Repräsentieren einzelne Zeichen als Vektoren
- Wort ist eine Sequenz aus Zeichen-Embeddings
- Wort-Vektor entsteht durch Weiterverarbeitung von Char Embeddings
  - CNNs
  - RNNs, LSTMs, GRUs
  - Pooling



Ling, W., Luís, et al. (2015). Finding Function in Form: Compositional Character Models for Open Vocabulary Word Representation. EMNLP.
Zhang X., et al. (2015). Character-level convolutional networks for text classification. In Proceedings NIPS

Byte Pair Encoding

- Subword-Tokenisierungsverfahren
- Ursprünglich aus der **Datenkompression**, später für NLP adaptiert
- Ziel:
  - Kompromiss zwischen **Word-Level** und **Character-Level**
  - **Offenes Vokabular** ohne echtes OOV-Problem
- Word-Level:
  - OOV-Wörter
  - großes Vokabular
- Character-Level:
  - sehr lange Sequenzen
  - Semantik schwer zu lernen
- Byte Pair Encoding:
  - kontrollierte Vokabulargröße
  - produktive Morphologie
  - robuste Repräsentationen
  - Einsatz als unsupervised Text-Tokenizer

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural MachineTranslation of Rare Words with Subword Units. ACL 2016.

Byte Pair Encoding

1. Initialisiere Vokabular mit allen Zeichen aus dem Text
2. Füge dem Vokabular das häufigste N-Gramm hinzu
3. Wiederhole 2, bis Vokabulargröße erreicht wurde

BPE lernt, welche Wortteile ”sich lohnen”, eigene Tokens zu werden.

Google WordPiece Modell ist eine Erweiterung

Byte Pair Encoding – Beispiel

1. Initialisiere Vokabular mit allen Zeichen aus dem Text

Byte Pair Encoding – Beispiel

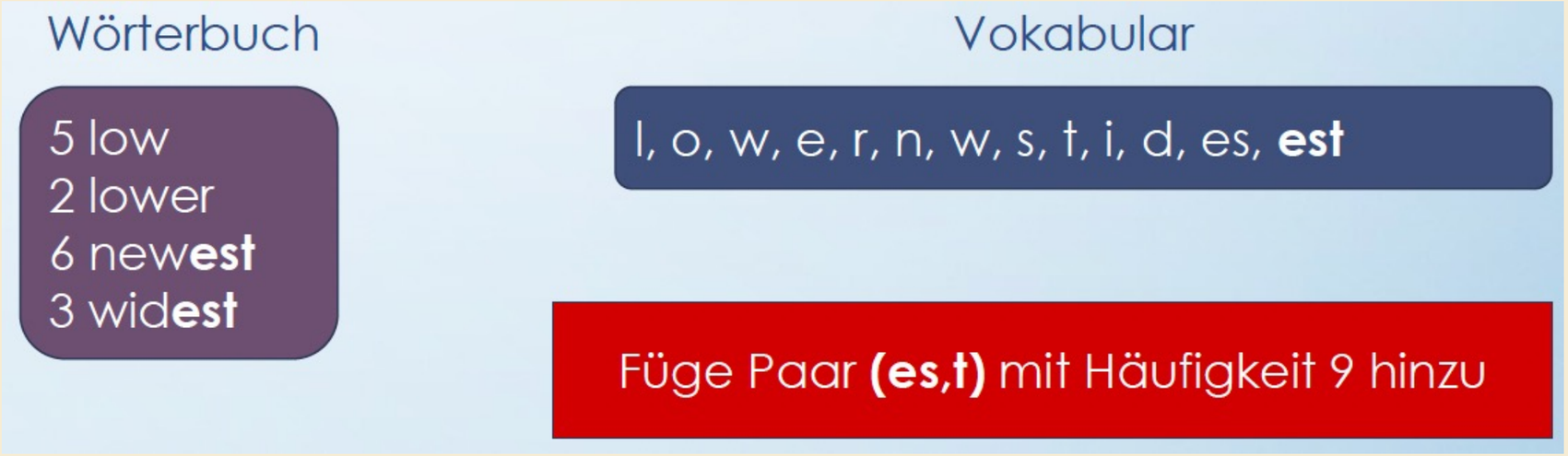
1. Initialisiere Vokabular mit allen Zeichen aus dem Text
2. Füge dem Vokabular das häufigste N-Gramm hinzu

(1)

Byte Pair Encoding – Beispiel

1. Initialisiere Vokabular mit allen Zeichen aus dem Text
2. Füge dem Vokabular das häufigste N-Gramm hinzu

(2)



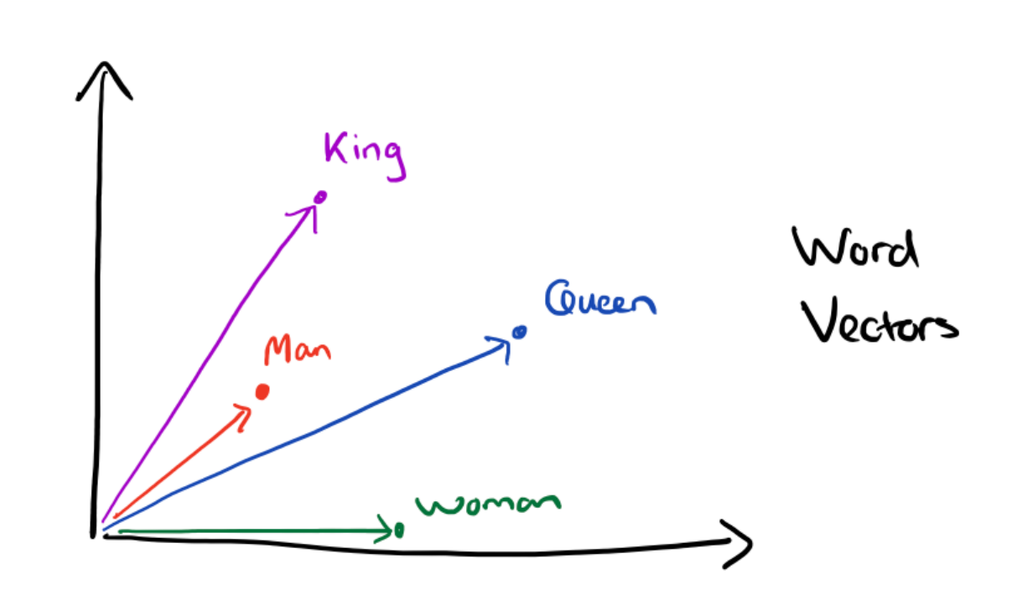
Byte Pair Encoding – Beispiel

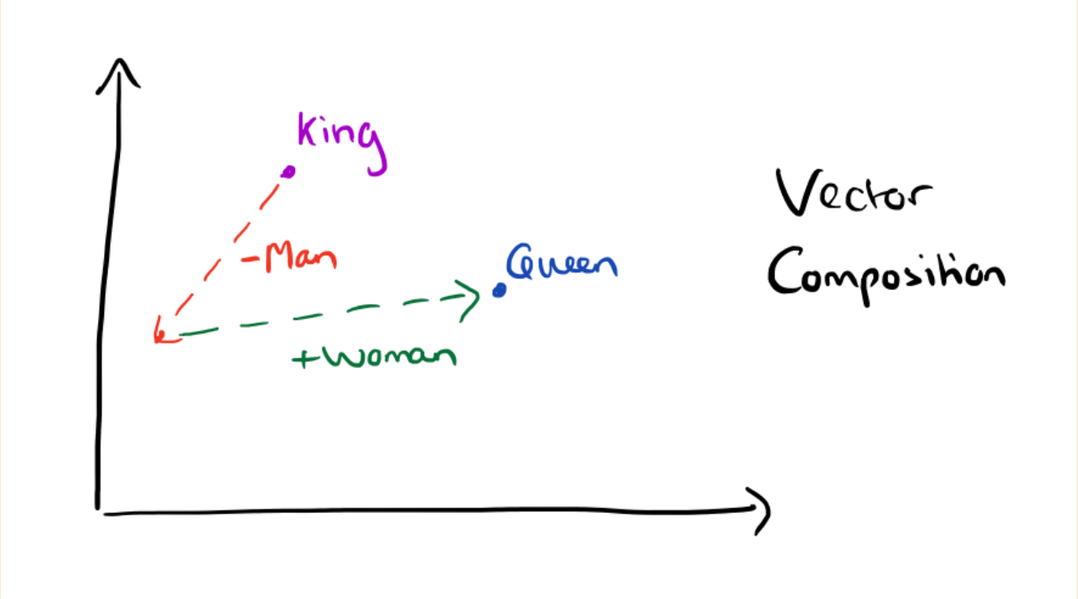
1. Initialisiere Vokabular mit allen Zeichen aus dem Text
2. Füge dem Vokabular das häufigste N-Gramm hinzu

(3)

Übung: Word-Embeddings

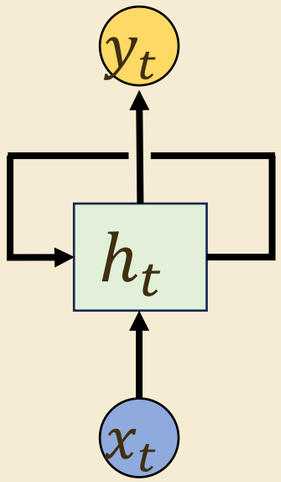
- Notebook: **`00_word_embeddings.ipynb`**
- Word-Embeddings (Vektoren) eignen sich zur Beantwortung von Analogiefragen der Form A ist zu B wie C zu ???





https://blog.acolyer.org/2016/04/21/the-amazing-power-of-word-vectors/

Recurrent Neural Networks



Warum rekurrente Netze?

Beispiel – Text:

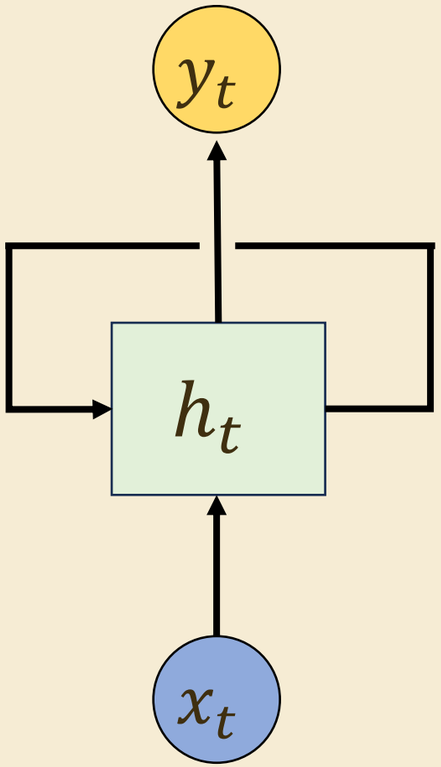
Wenn wir einen Satz in einem Text lesen, wird mit jedem neuen Wort unser **aktueller Informationsstand** aktualisiert, und er hängt nicht nur von dem **neubeobachteten Wort** ab, sondern auch von den **vorhergehenden Wörtern**.

Allgemein:

- Viele Daten sind sequenziell strukturiert
- Reihenfolge trägt semantische Information
- Klassische Netze ignorieren Zeitabhängigkeiten
- Weiteres Beispiel: Zeitreihen
- Ziel: Modellierung von Abhängigkeiten über Zeit

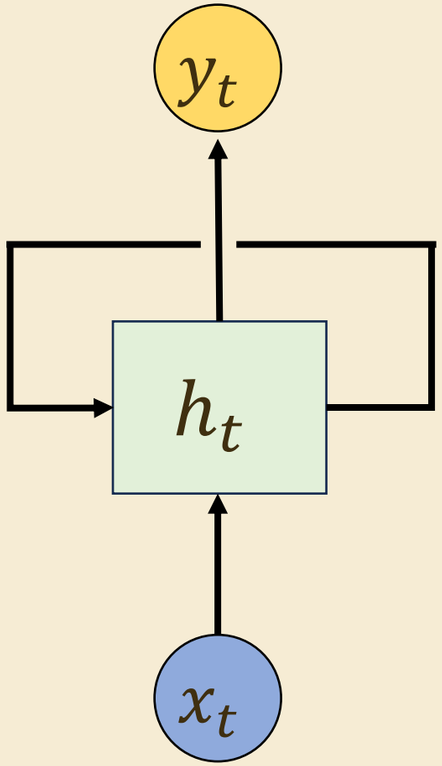
RNN-Architektur

- Ein Teil des Outputs ist gleichzeitig Teil des Inputs:



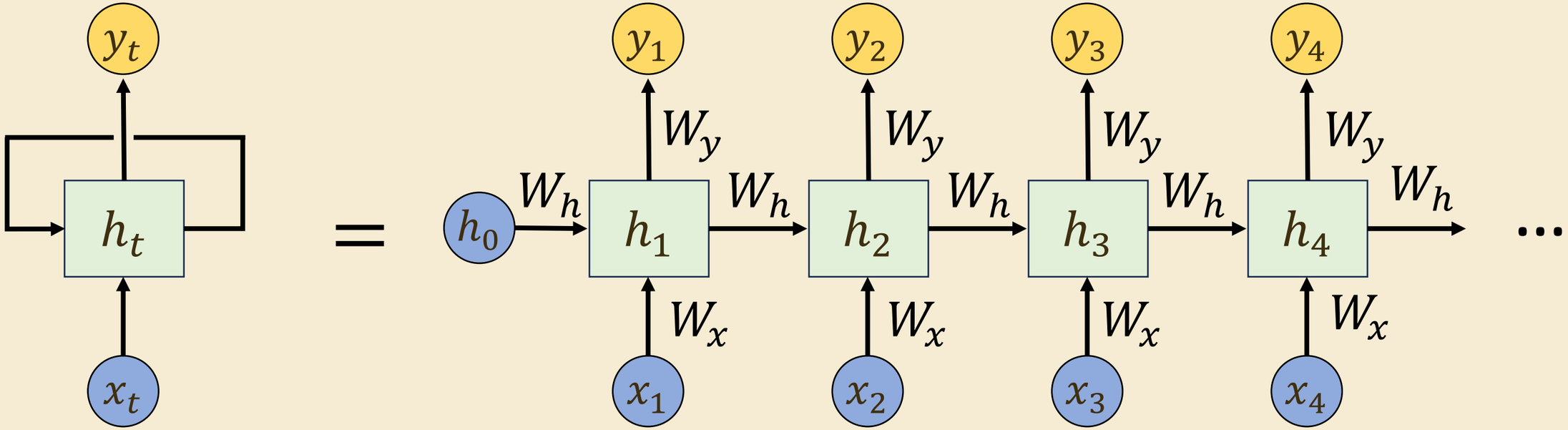
Intuition: Ein Netz mit Gedächtnis

- Gleiche Transformation bei jedem Zeitschritt
- Interner Zustand speichert Vergangenheit
- Eingabe beeinflusst aktuellen Zustand
- Zustand beeinflusst zukünftige Ausgaben
- „Gedächtnis“ ist Teil des Modells



(Vanilla) RNN-Architektur

- Tatsächliche Implementierung durch ”Ausrollen”:



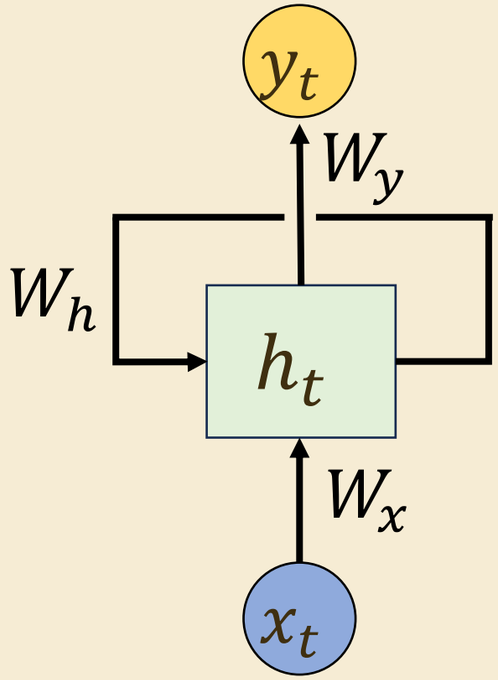
$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$$

$$y_t = \mathrm{g}(W_y \cdot h_t + b_y)$$

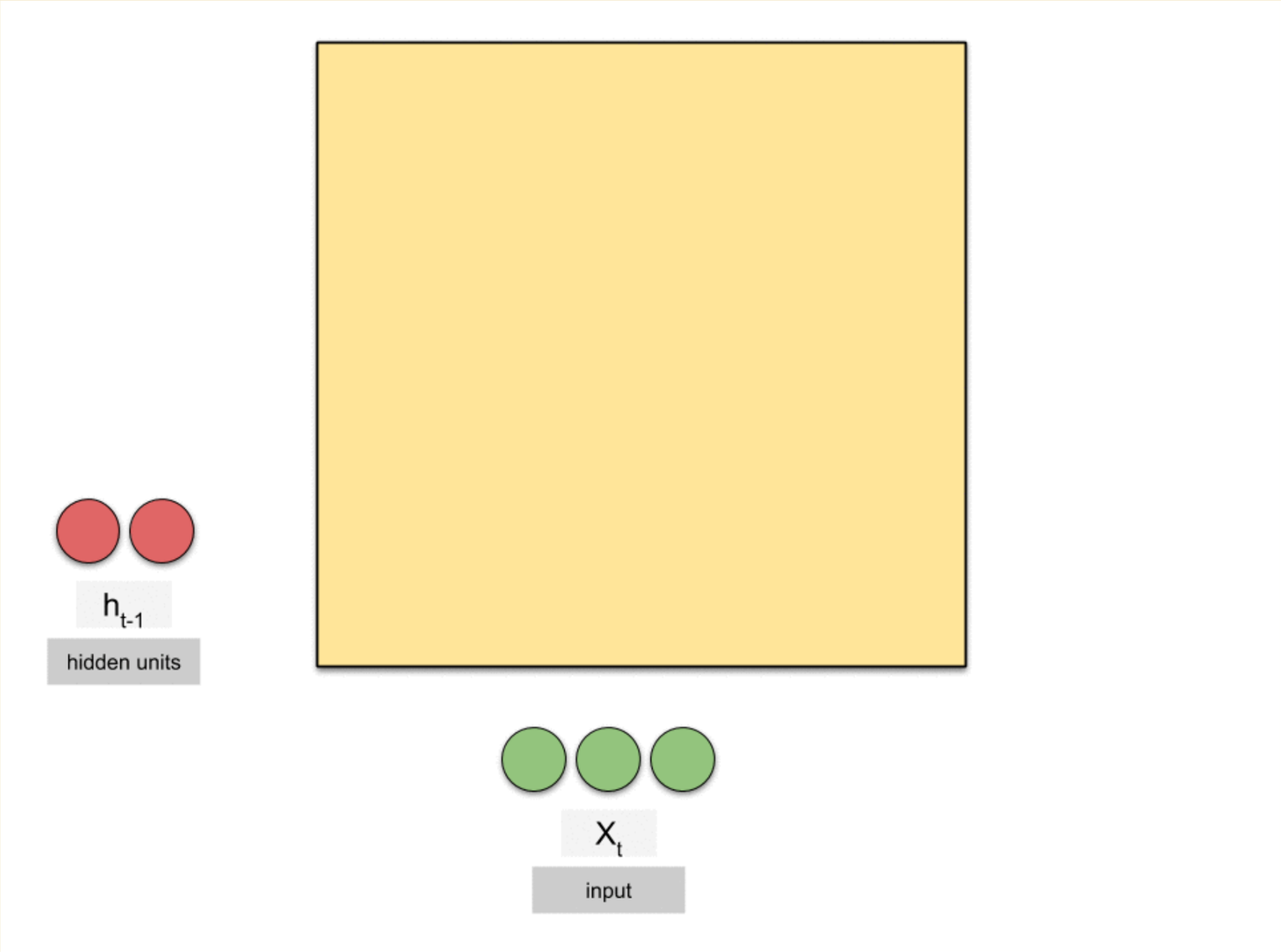
g: Aktivierungsfunktion für Output (z.B. Softmax oder linear)

(Vanilla) RNN-Architektur

- $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$
  - $\tanh$, weil Wertebereich $(-1; 1)$
- $y_t = \mathrm{g}\left(W_y \cdot h_t + b_y\right)$, g: Aktivierungsfunktion für Output (z.B. Softmax oder linear)
- x_t: Eingabe zum Zeitpunkt t
- h_t: Hidden State
- W_x, W_h, b_h, b_y: lernbare Parameter



(Vanilla) RNN Animation



https://medium.com/data-science/animated-rnn-lstm-and-gru-ef124d06cf45

Vergleich: Vanilla RNN und MLP

- (Vanilla) RNN-Zelle entspricht einem normalen MLP-Layer mit den Inputs (x_t, h_{t-1}) und zwei Outputs (y_t, h_t)
- Allerdings werden für y_t und h_t unterschiedliche Aktivierungsfunktionen verwendet (y_t: linear oder Softmax, h_t: meist tanh)
- Das zeitliche Ausrollen entspricht einem tiefen MLP mit Weight Sharing (also gleiche Gewichte in allen Layern)

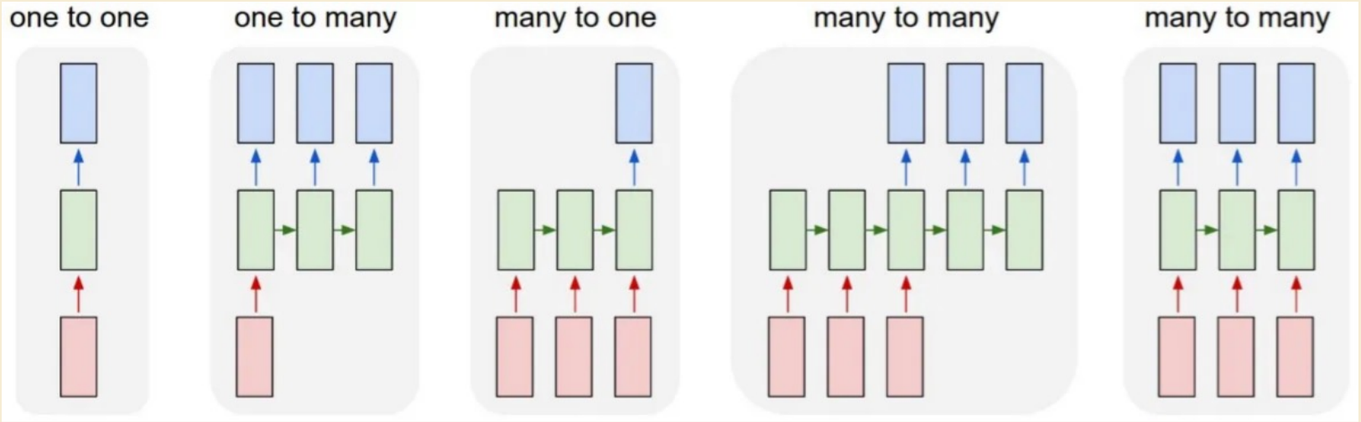
Initialer hidden state vector ($\boldsymbol{h_0}$)

Möglichkeiten:

- Typischerweise: $h_0 =$ Null-Vektor
- Lernbare Parameter – insbesondere, wenn die Anwendung sehr kurze Sequenzen hat
- Beim Training auf sehr langen Sequenzen (z.B. ganzes Buch, lange Zeitreihe), die in viele kurze Sequenzen unterteilt worden sind, kann der Endzustand des vorherigen Batches genutzt werden – so kann das RNN sich an diesen Teil "erinnern" (Keras: `stateful=True`)

RNN-Eigenschaften

- Verarbeitet **beliebig lange Sequenzen**
- Parameterzahl unabhängig von Inputlänge
- Berechnung langsam, **schlecht parallelisierbar aufgrund sequentieller Struktur**
- Ermöglicht vielfältigen Einsatz
  - Tagging (POS, NER)
  - Klassifikation (Sentiment)
  - Question Answering durch Encoding
  - Machine Translation



Backpropagation through time

- Jeder Zeitschritt einer RNN-Sequenz kann zum Loss beitragen:

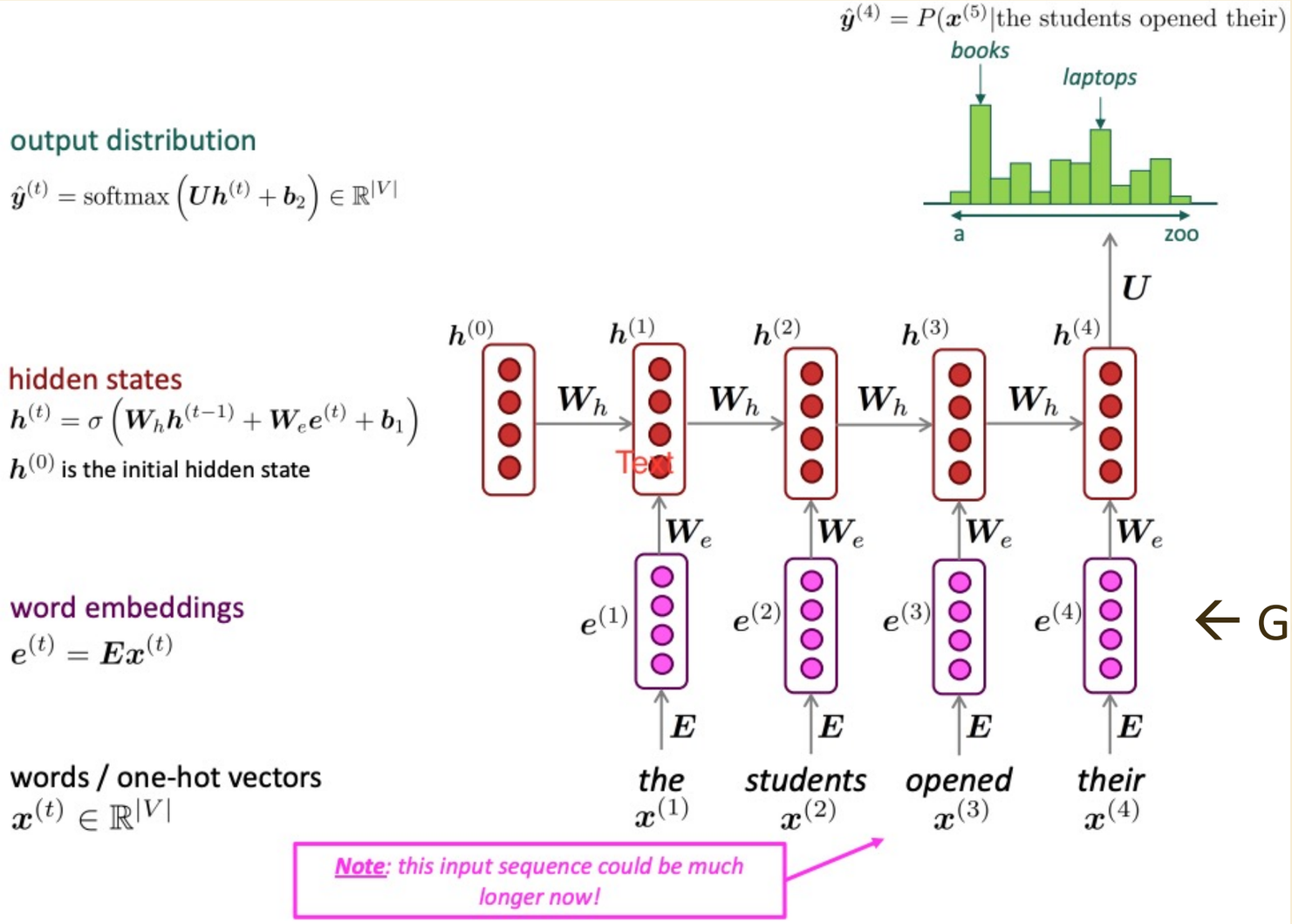
$$\text{Lossfunktion:} \quad L = \sum_{t=1}^{T} l\,(y_t, \hat{y}_t)$$

- L: loss, y_t: Label zum Zeitpunkt t, $\hat{y}_t$: Vorhersage
- Die Parameter sind in jedem Zeitschritt gleich
  - "weight sharing through time"
- Durch Abhängigkeit von h_{t-1} beeinflussen die Parameter sämtliche Zeitschritte
- Die Berechnung der Gradienten in verschiedenen Zeitschritten und Update der Parameter bezeichnet man als "Backpropagation through time"

Truncated backpropagation through time

- Lange Sequenzen → Gradienten über sehr viele Zeitschritte
  - Berechnung rechenintensiv
  - Zeitschritte sehr weit hinten tragen oft wenig bei (geringe Gradienten)
- Daher wird beim Training oft die Länge der Sequenzen begrenzt
- Backprop in Frameworks (TensorFlow, PyTorch) läuft über gesamte Sequenzlänge – Sequenzen manuell kürzen, wenn gewünscht

RNN und Language Models



← Gelernte Word Embeddings

https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1246/slides/cs224n-spr2024-lecture05-rnnlm.pdf (S. 32)

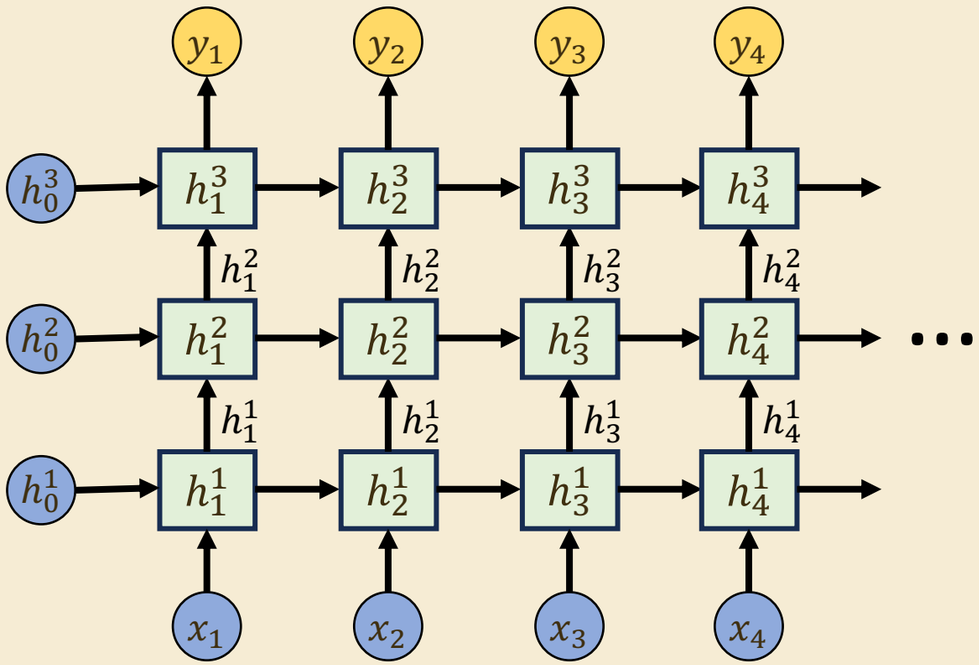
RNN mit Keras

Keras:

- SimpleRNN: $y_t = h_t$
- Vanilla-RNN mit separatem Output lässt sich über Einfügen eines expliziten Dense-Layers realisieren

Stacked RNN

- Analog zum Hinzufügen weiterer Layer in einem MLP
- Erhöht Modellkapazität



$$h_t^1 = \tanh(W_x^1 \cdot x_t + W_h \cdot h_{t-1}^1 + b_h^1)$$
$$h_t^2 = \tanh(W_x^2 \cdot h_t^1 + W_h \cdot h_{t-1}^2 + b_h^2)$$
$$\ldots$$

Anzahl an Layern abhängig von Anwendung

- Zeitreihen: 1-2
- Sentiment Analysis: 2-3
- Spracherkennung: bis 8
- Machine Translation: bis 8

RNN – Probleme

- Informationen vor ”vielen” Zeitschritten schwer zugänglich
  - Vanishing Gradient
  - Exploding Gradient
  - Dadurch “vergisst” das Netzwerk den Beginn langer Sequenzen und kann keine langfristigen Abhängigkeiten lernen
- Beschränktes Gedächtnis (“Information Bottleneck”)
  - Hidden state h_t dient sowohl als Output Repräsentation als auch als gesamtes Memory
  - neue Information überschreibt oft die alte: $h_t = \tanh(W_x x + W_h h_{t-1} + b_h)$
  - Kein Mechanismus zum gezielten Behalten oder Vergessen von Information
- Berechnung langsam, da kaum parallelisierbar
  - h_{100} kann erst berechnet werden, wenn Berechnungen für $t = 99$ fertig sind usw.
  - Daher nur langsam trainierbar und wenig Speedup durch moderne GPUs, die für massive Parallelität ausgelegt sind

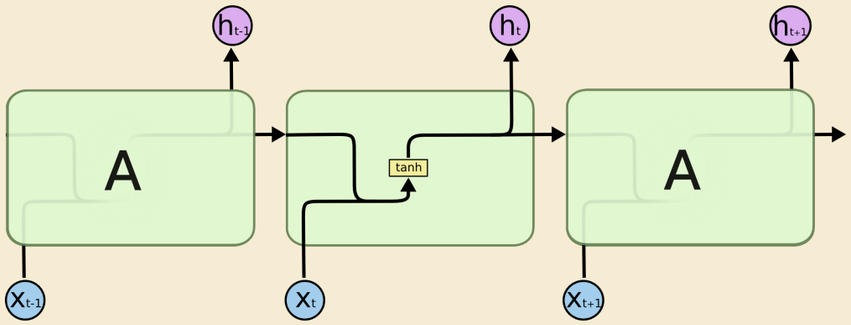
LSTM: Long short-term memory

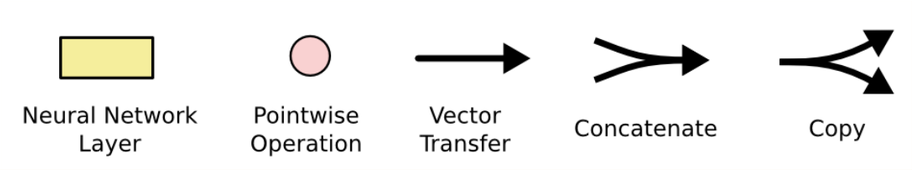
- Fortgeschrittene RNNs unterscheiden sich vor allem dadurch, wie der hidden state aktualisiert wird
- RNN-Erweiterung zur Reduktion des ”vanishing Gradient” Problems
- LSTM hat einen expliziten ”Memory Mechanismus”
  - Erlaubt es Informationen über mehr Schritte zu behalten
- Modell lernt, welche Informationen
  - Gespeichert
  - Gelöscht
  - Ausgelesen und für das Update genutzt werden

Sepp Hochreiter, Jürgen Schmidhuber; Long Short-Term Memory. *Neural Comput* 1997; 9 (8): 1735–1780. doi: https://doi.org/10.1162/neco.1997.9.8.1735

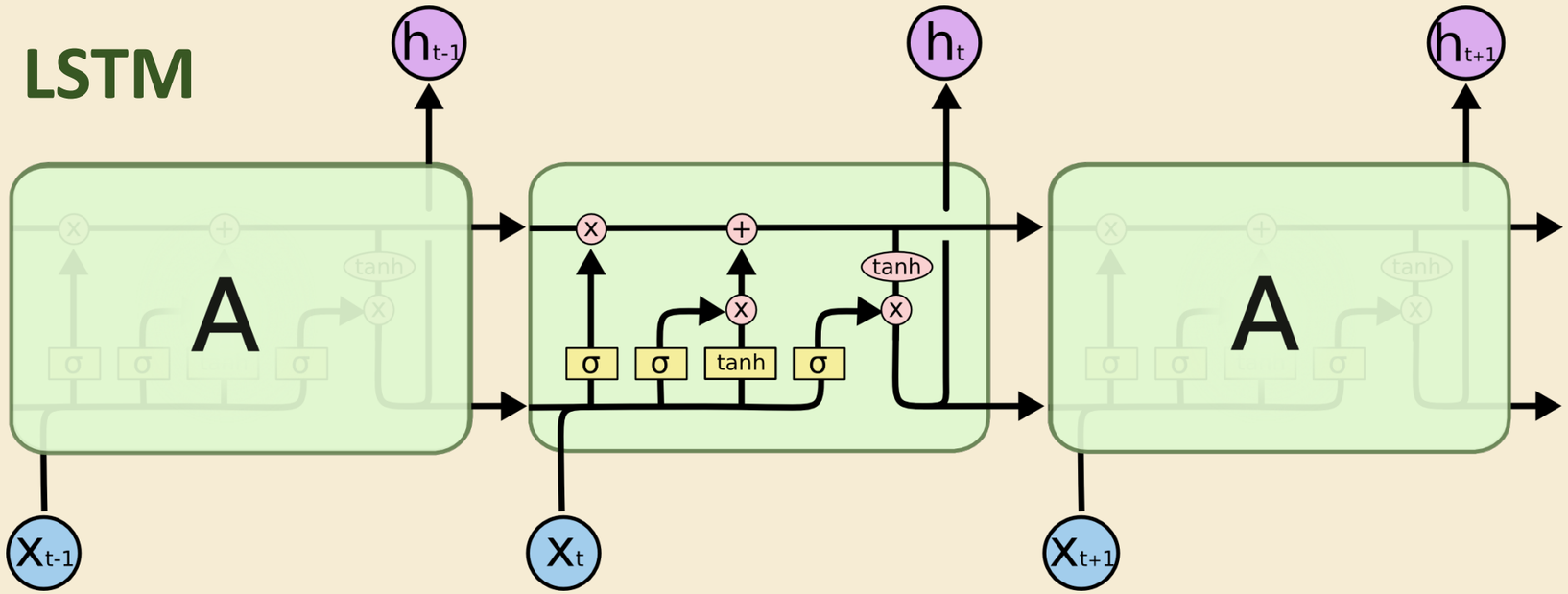
LSTM

Vanilla RNN





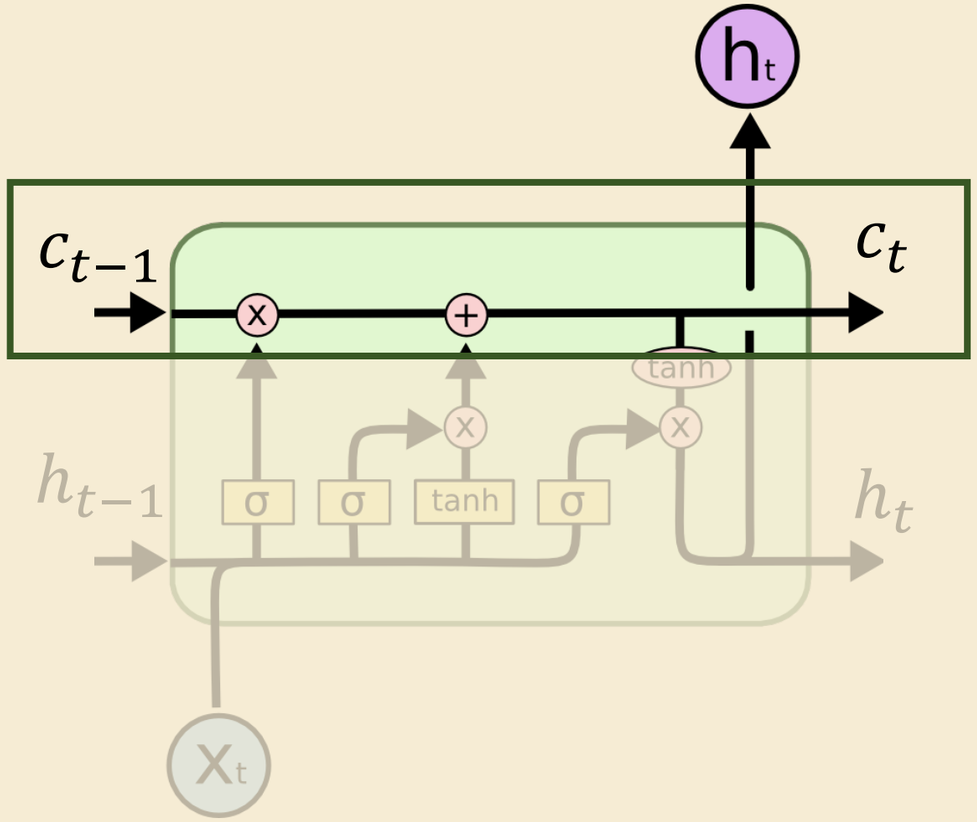
LSTM



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

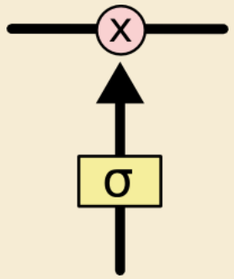
LSTM – Kernidee: Cell State

- LSTM-Kern: Zellzustand c_t (horizontale Linie oben im Diagramm)
- Förderband entlang der gesamten LSTM-Kette
- Information kann leicht unverändert transportiert werden
- LSTM kann Informationen hinzufügen oder entfernen
- Reguliert durch sog. “Gates”

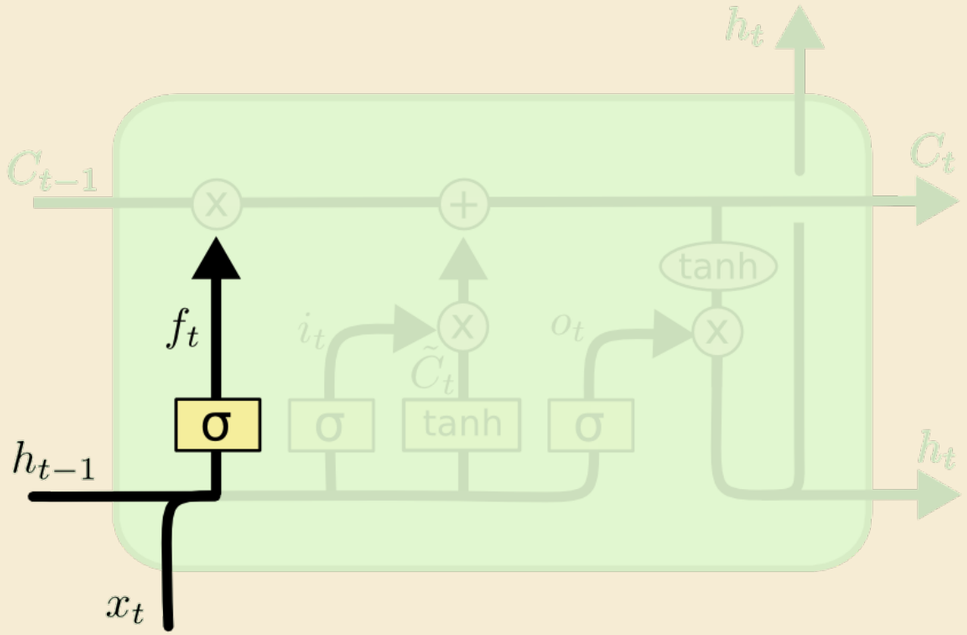


Funktionsweise Gates

- Gates können ausgewählte Informationen durchlassen oder entfernen
- Mathematische Operation: Multiplikation mit Sigmoid $\sigma \in (0; 1)$
  - Wertebereich geeignet zum Durchlassen (1) oder Löschen (0) durch Multiplikation
- Input und Parameter der Sigmoid-Funktion bestimmen, welche Informationen behalten und welche gelöscht werden



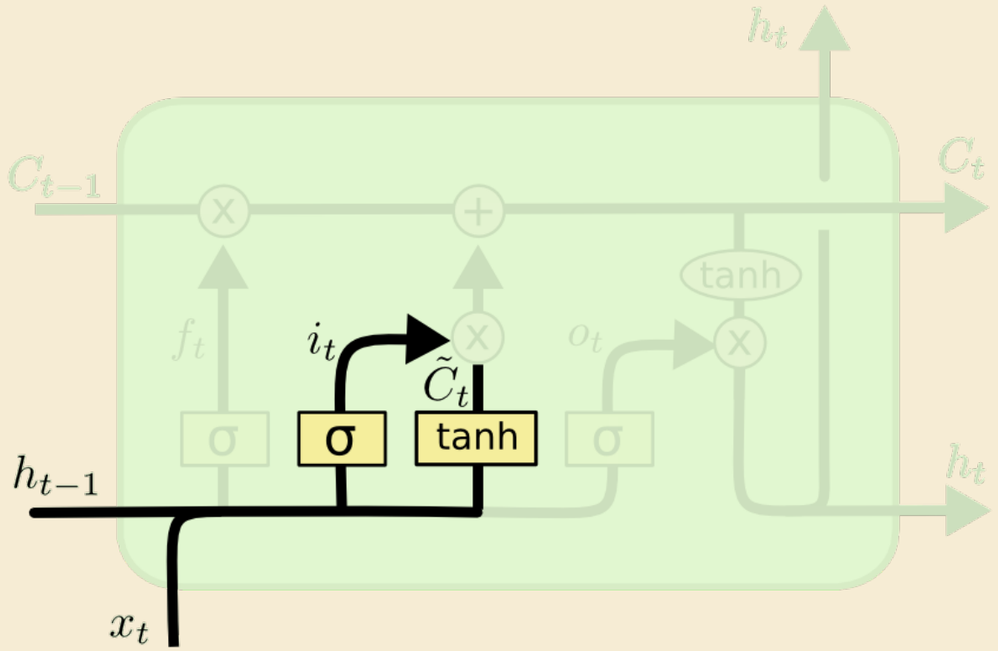
LSTM Schritt für Schritt (1)



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

- Schritt 1: Welche Information wird behalten (“forget gate”)?

LSTM Schritt für Schritt (2)

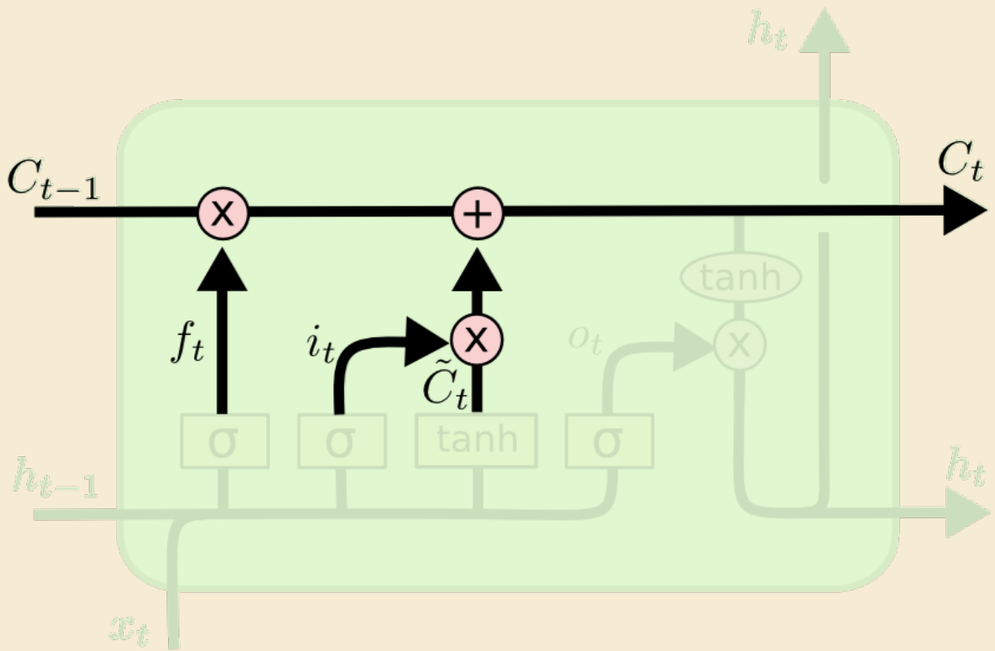


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

- Schritt 2: Welche neuen Informationen hinzufügen (“input gate”)?
- Tanh berechnet potentiell hinzuzufügende Informationen
- Sigmoid σ bestimmt, welche davon zum Zellzustand c_t hinzugefügt werden

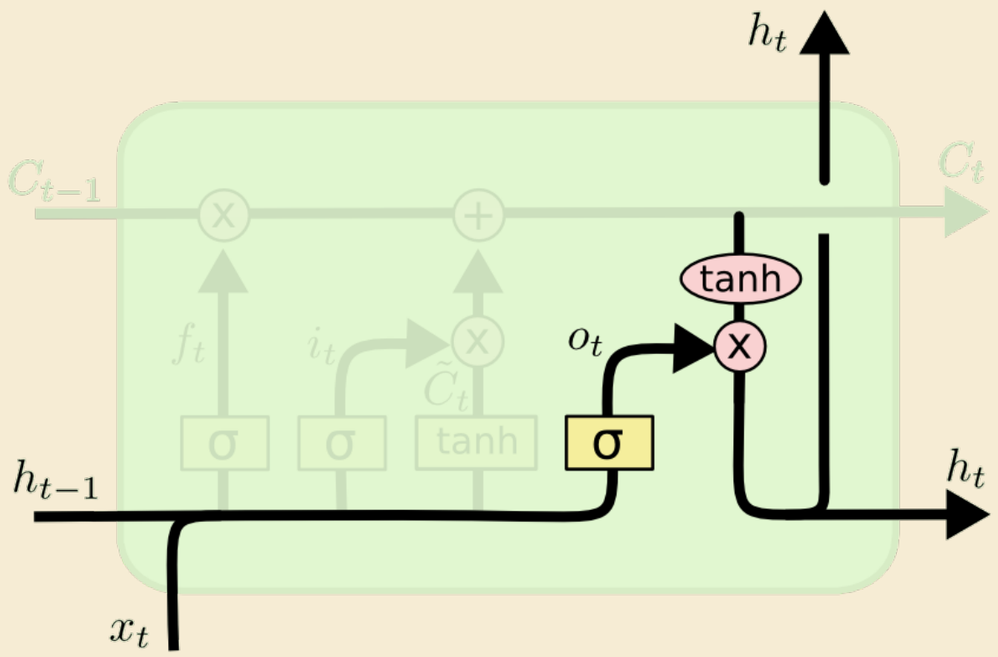
LSTM Schritt für Schritt (3)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Schritt 3: Update des alten Zellzustands C_{t-1}
- Multiplikation von C_{t-1} mit f_t bestimmt, was vergessen wird
- $\widetilde{C}_t$ enthält neue Informationen, Multiplikation von $\widetilde{C}_t$ mit i_t entscheidet, welche davon durch Addition zum neuen Zellzustand C_t hinzugefügt werden

LSTM Schritt für Schritt (4)



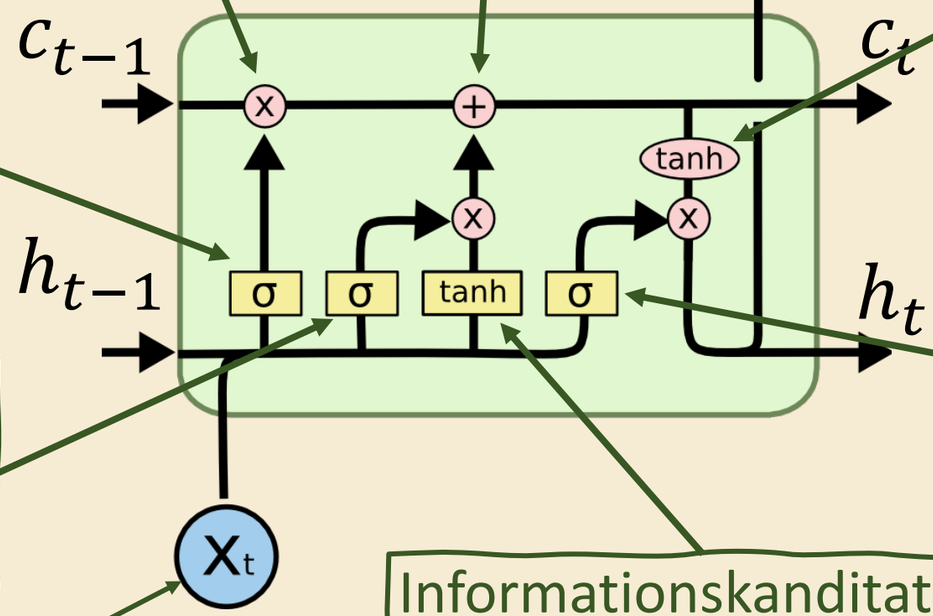
$$o_t = \sigma\left(W_o\ [h_{t-1}, x_t]\ +\ b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

- Schritt 4: Bestimmung des Outputs bzw. neuen hidden states h_t
- Output (neuer hidden state h_t) basiert auf Cell State C_t - es wird aber gefiltert
- Filter o_t berechnet sich aus bisherigem hidden state h_{t-1} und aktuellem Input x_t
- Cell state wird noch durch tanh transformiert, damit alle Werte in (-1;1) sind

LSTM-Komponenten

Wertebereiche
σ: (Sigmoid) $\in (0; 1)$
tanh: $\in (-1; 1)$



input gate

hidden state

forget gate

tanh transformiert cell state in (-1;1) vor dessen Verwendung als hidden state

Berechne forget gate aus x_t und h_{t-1}

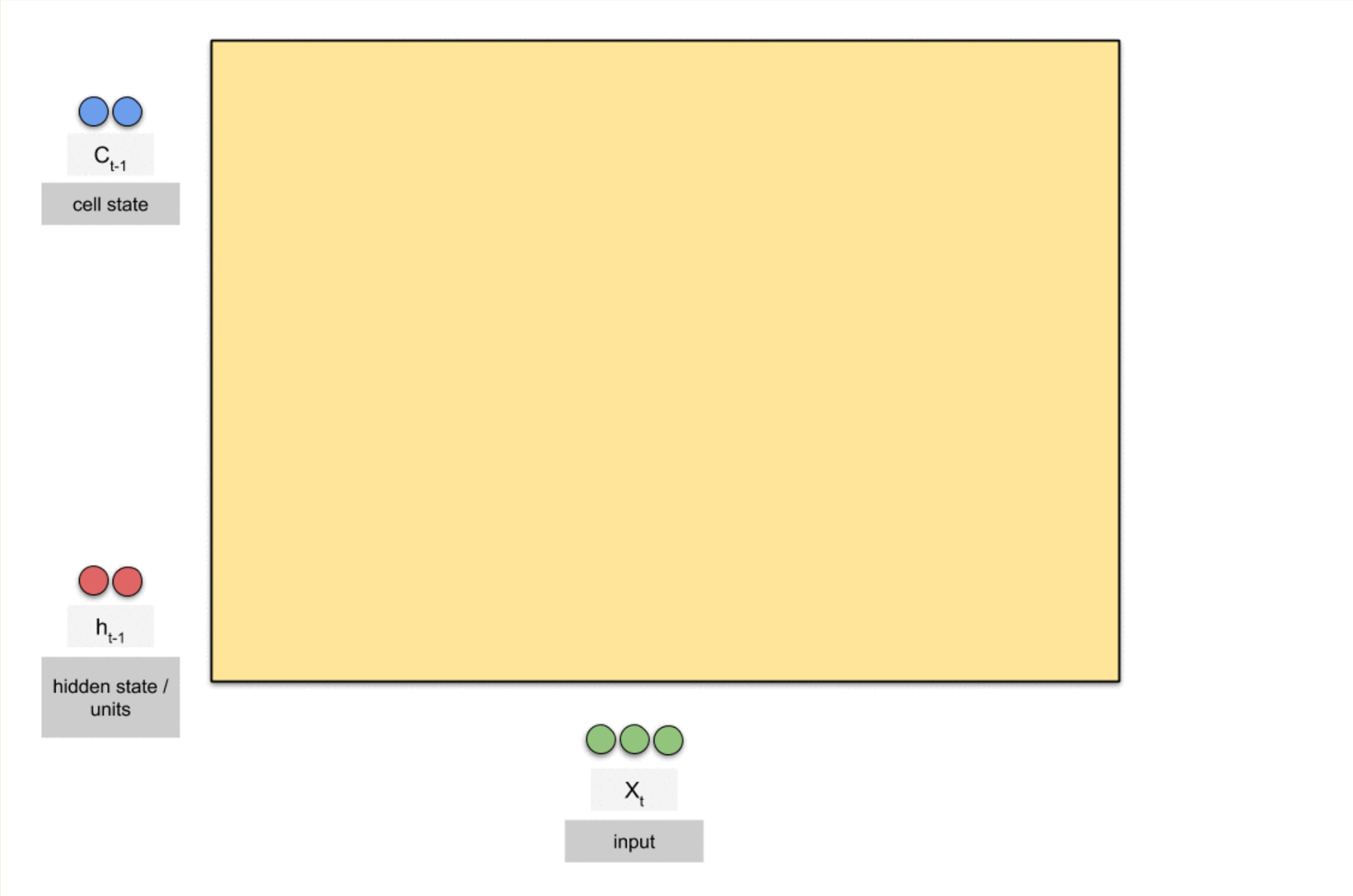
Steuert abhängig von x_t und h_{t-1}, welche Infos aus cell state c_t in den hidden state h_t fließen

Welche der Informationskandidaten wie stark hinzufügen

Informationskanditaten zum Hinzufügen

input

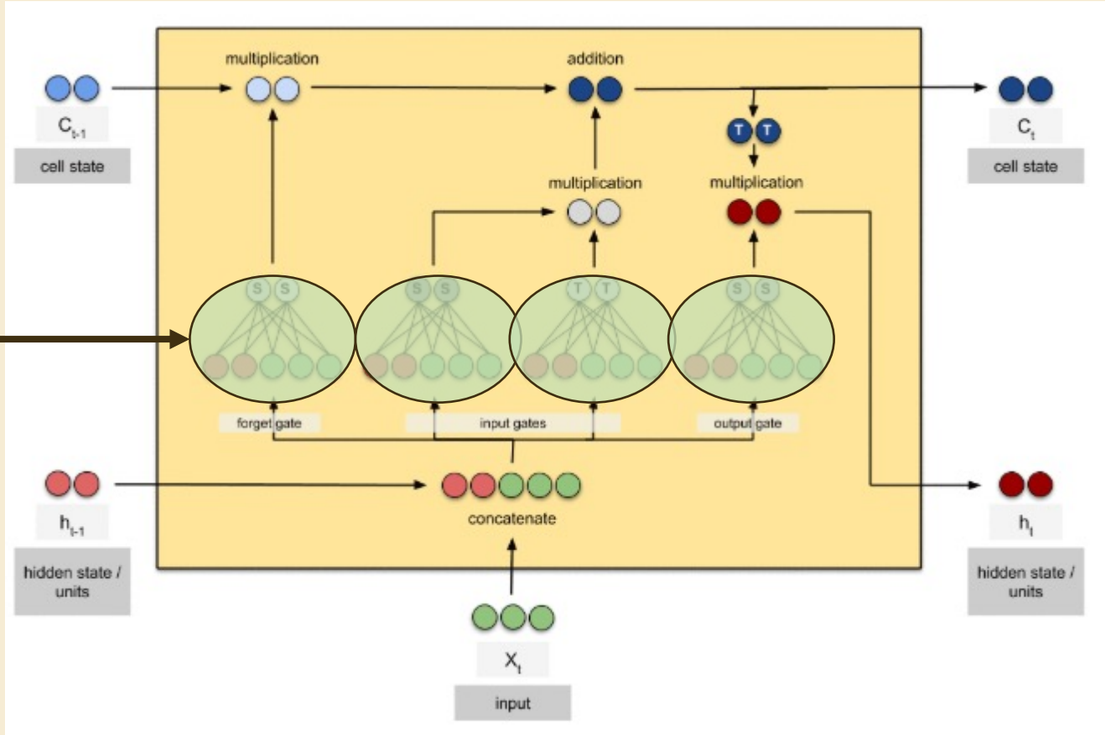
LSTM-Animation



https://medium.com/data-science/animated-rnn-lstm-and-gru-ef124d06cf45

LSTM-Parameter zählen

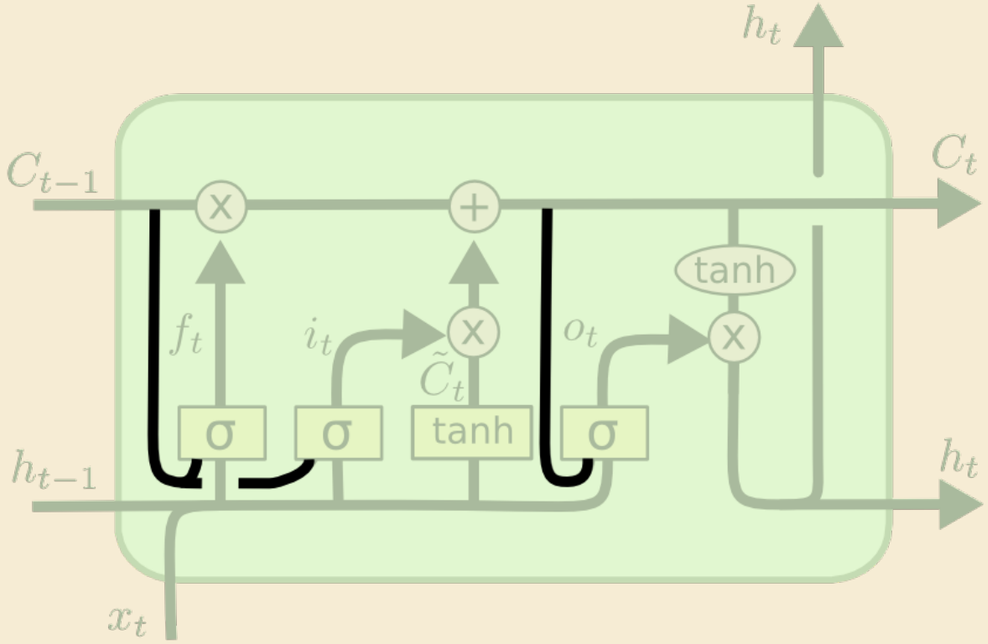
`params_per_gate`



- x: Input-Dimension
- h: Hidden-Dimension
- $\text{params_per_gate} = (x + h) * h + h$
- 4 Gates → $\#\text{params}_{total} = 4 * \big((x + h) * h + h\big)$

LSTM-Varianten (1)

- Es gibt zahlreiche LSTM-Varianten
- Variante 1: “Peephole Connections” (Gate Layer hängen ebenfalls von Cell State ab):



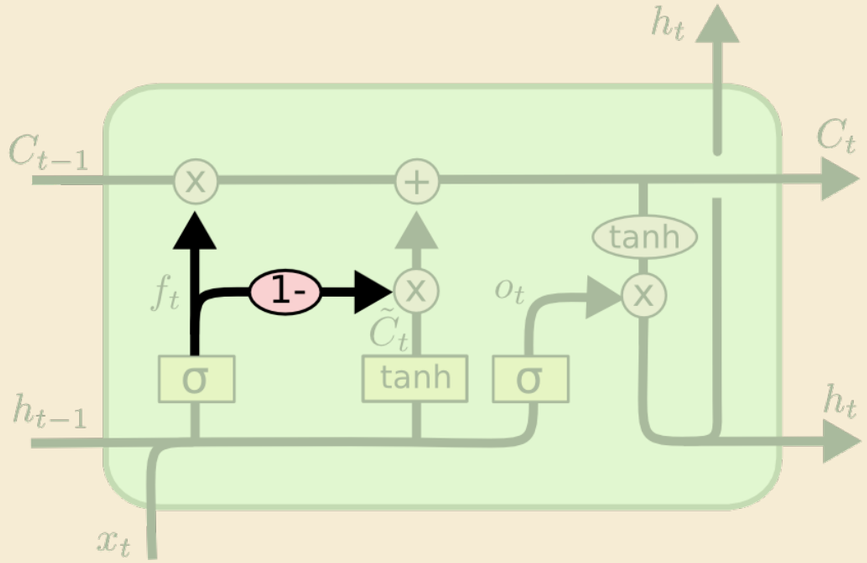
$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

Hier: Peepholes in allen Gates; aber auch Auswahl möglich

Engl. Peephole: Guckloch

LSTM-Varianten (2)

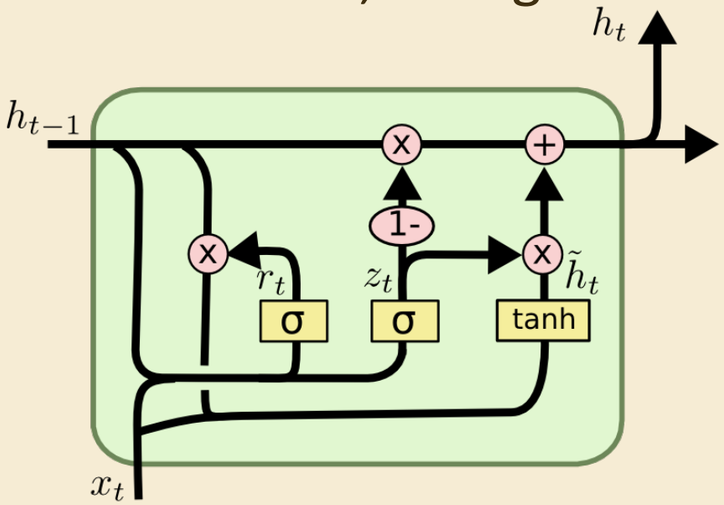
- Variante 2: gekoppelte Forget- und Input-Gates (“Update Gate”)
- Was vergessen wird, wird durch neue Informationen ersetzt und umgekehrt werden nur neue Informationen an Stellen eingefügt, die vergessen wurden:



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

GRU: Gated Recurrent Unit

- Kann man auch als (populäre) LSTM-Variante betrachten
- Update Gate statt separater Forget- und Input-Gates
- Reset Gate
- Kein separater Cell State
- Schneller trainierbar, weniger Parameter



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right) \longleftarrow \text{Update Gate}$$

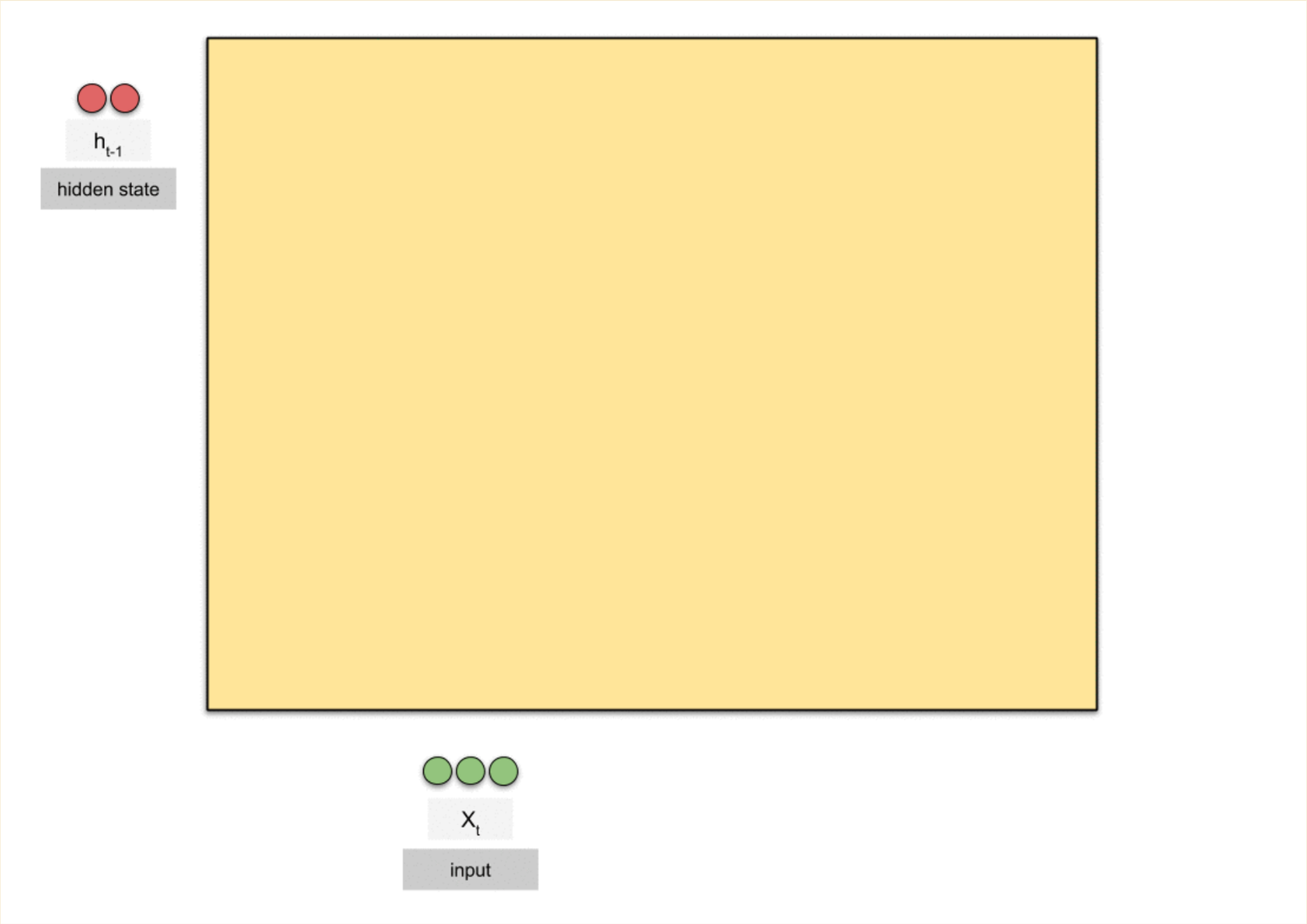
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right) \longleftarrow \text{Reset Gate}$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Cho et al. 2014. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics. https://arxiv.org/abs/1406.1078

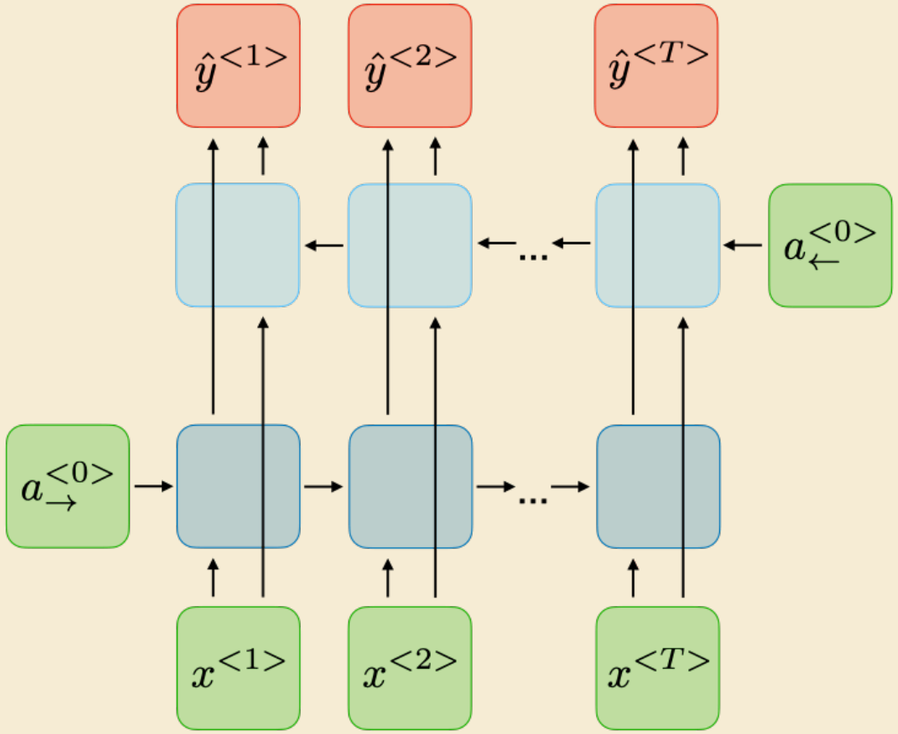
GRU-Animation



https://medium.com/data-science/animated-rnn-lstm-and-gru-ef124d06cf45

Erweiterung: Bi-directional LSTM (oder GRU)

- Bisher: LSTM kennt nur den linken Kontext
- Können zweites LSTM (GRU) einfügen, welches die Sequenz von rechts nach links verarbeitet
- Konkatenieren Output beider LSTMs (GRUs)



https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

Vanishing/Exploding Gradients

- Häufiges Problem bei sehr langen Sequenzen (oder tiefen Netzen)
- Gradienten werden immer kleiner → kein Parameter-Update
- Gradienten können auch immer größer werden → sehr große Updates, Loss divergiert
- Kein reines RNN Problem
  - Auch tiefe CNNs und MLPs betroffen
  - Bei RNNs besonders schlimm, da Gradient je nach Sequenzlänge sehr weit zurückpropagiert werden muss (>100)

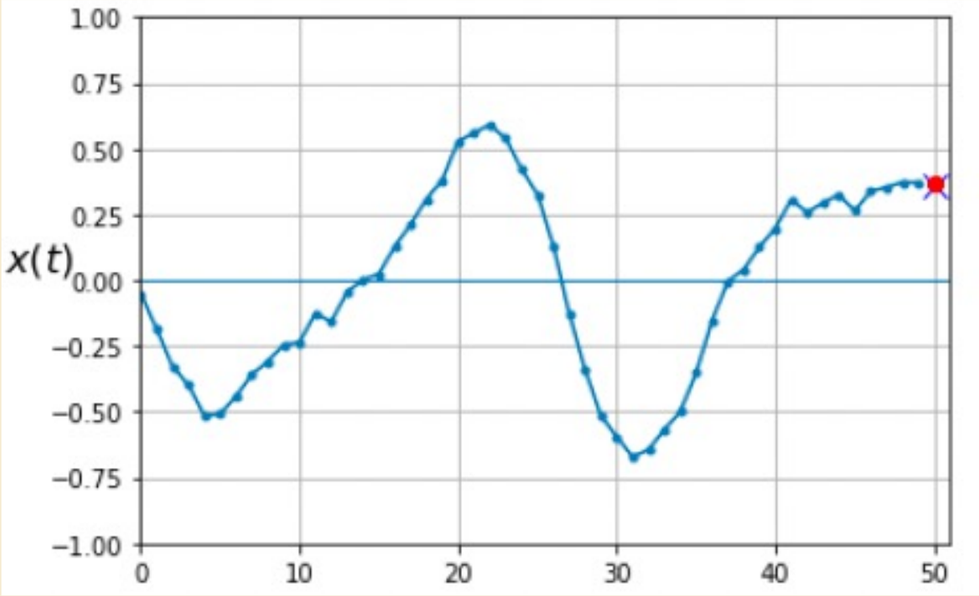
Vanishing/Exploding Gradients – Lösungen

- Glorot und Bengio (2010) zeigten, dass Initialisierung und Aktivierungsfunktion relevant sind
  - “Glorot-Initialisierung”
  - Keine logistische Aktivierungsfunktion in den hidden layern
- Batch-Normalisierung
- Gradienten-Clipping: Updates-Größe manuell begrenzen
- Spezialisierte Architekturen
  - Residual, Skip, Identity, Highway connections
- Langsames Erweitern des Netzwerks um weitere Schichten und Parameter einzelner Schichten nicht anpassen

Glorot, Xavier & Yere, Yere. (2010). Understanding the difficulty of training deep feedforward neural networks.

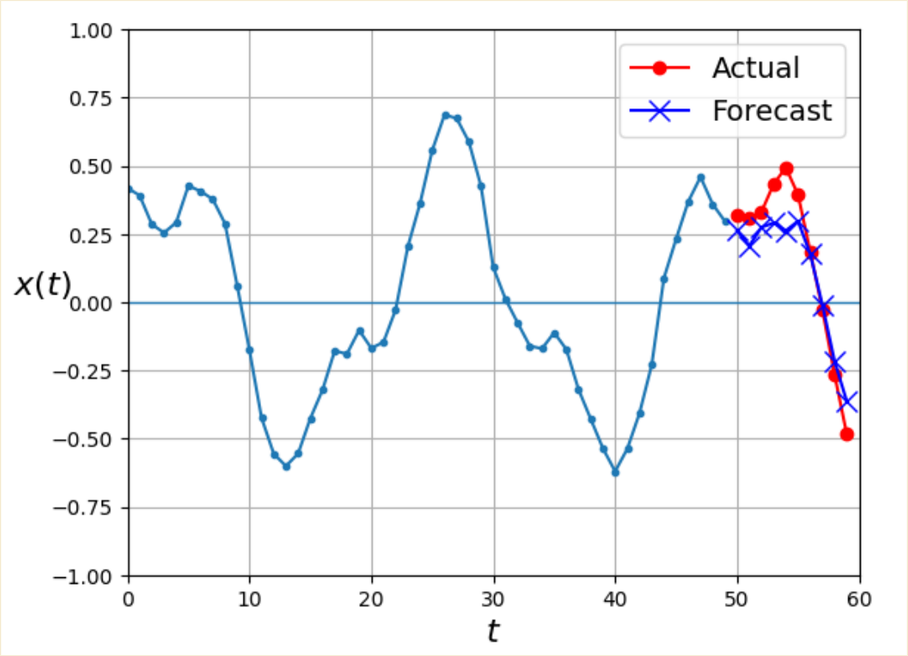
Übung 2

- Notebook: `01_VanillaRNN_timeSeries.ipynb`
- Künstlich erzeugte Daten zu Demozwecken
- 10,000 Zeitreihen mit je 51 Zeitschritten
- 7.000 train, 2.000 dev (validation), 1.000 test
- X(t): zwei überlagerte Sinuskurven + Rauschen
- Y: Wert der Kurve an Position 51



Übung 3

- Notebook: `02a_VanillaRNN_timeSeries-future.ipynb` & `02b_VanillaRNN_timeSeries-future.ipynb`)
- Möchten die **nächsten 10 Datenpunkte** vorhersagen (statt nur einen)
  - Zwei Varianten in Notebooks



Übung 4 – Language Modelling

- Language Model auf Shakespeare: shakespeare.ipynb
  - Trainingsziel: Vorhersage des nächsten Buchstabens
  - Nach dem Training können Texte im Shakespeare-Stil gesampelt werden
- ”The Unreasonable Effectiveness of Recurrent Neural Networks”, Andrej Karpathy, https://karpathy.github.io/2015/05/21/rnn-effectiveness/
  - Lesen und ausprobieren, hat Code auf Github (leider in LUA) und einen interessanten alternativen Datensatz

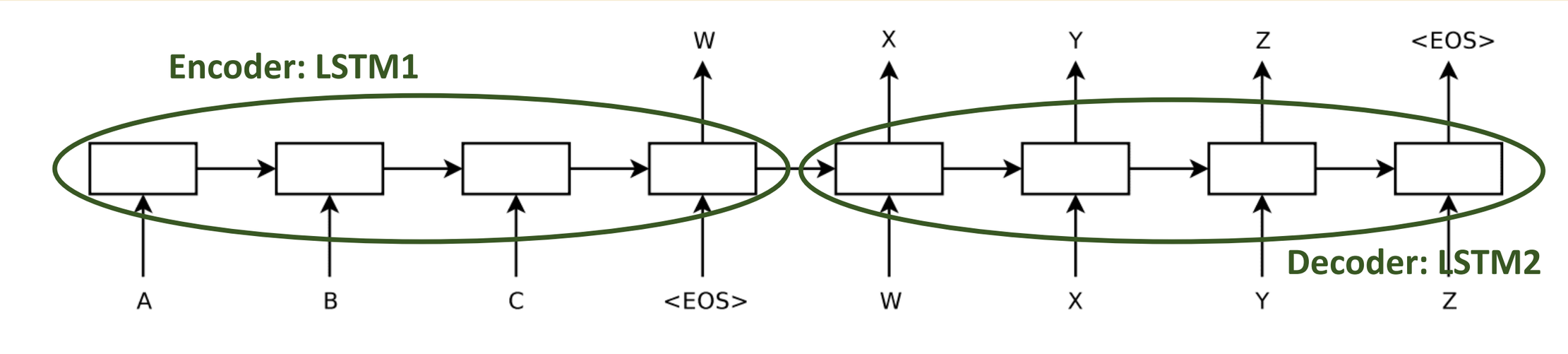
Übung 5

- Mehr als nur 10 Datenpunkte vorhersagen mit Modell, das für 10 Datenpunkte trainiert worden ist
- Mit anderen Modellarchitekturen (z.B. MLP) vergleichen
- Länger trainieren und optimieren
- Trainieren bis Overfitting einsetzt
- Training auf GPU (Zugangsdaten: Moodle)
- Implementieren Sie ein RNN mit PyTorch
- Sequence2Sequence/Encoder-Decoder implementieren

Seq2Seq & Encoder-Decoder-Architektur

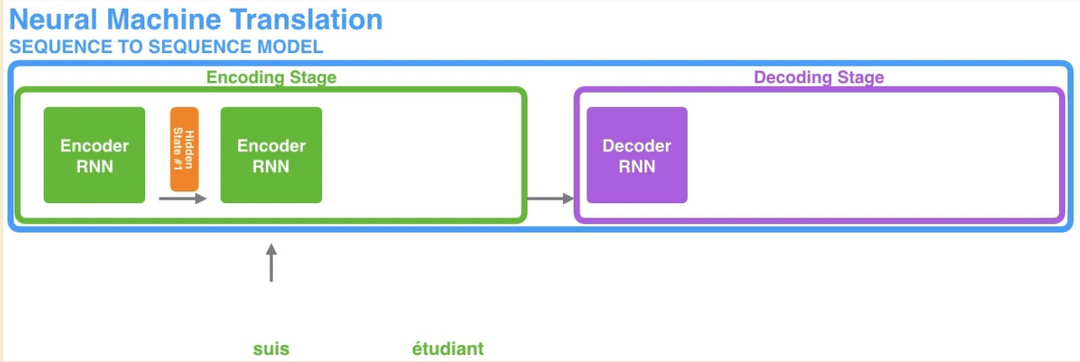
Sequence to Sequence Learning with Neural Networks, Sutskever et al, https://arxiv.org/abs/1409.3215

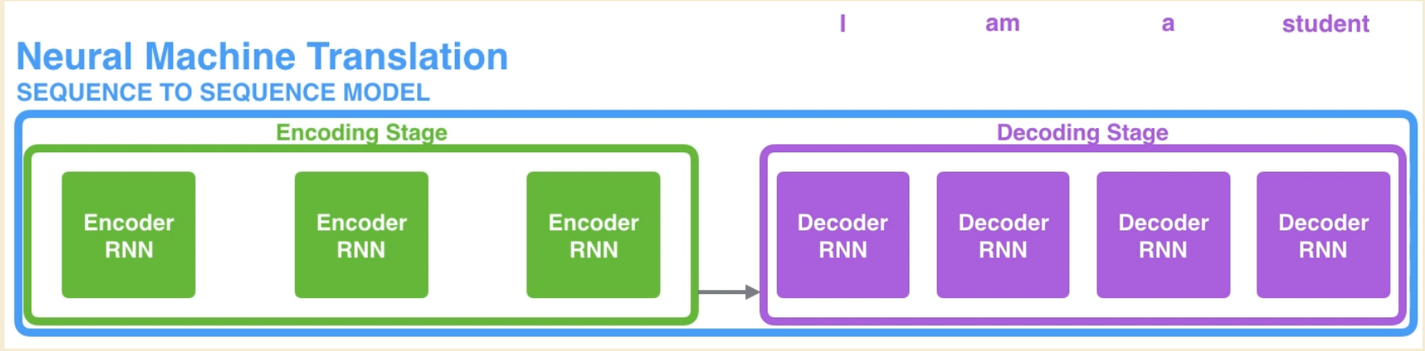
- Problemstellung: Machine Translation
  - Input: Satz in Englisch.
  - Target: Satz in Französisch.
- **Encoder** verarbeitet Input variabler Länge und transformiert ihn in Vektor definierter Größe.
  - Information steckt nun in diesem hidden state vector, ein Vektor fester Länge
  - Dieser dient als “previous hidden state” h_{t-1} des “Decoders”
- **Decoder** muss Sequenz variabler Länge vorhersagen
  - Basierend auf dem hidden state vector des Encoders
  - Input ist jeweils das zuvor generierte Wort des Decoders (zu Beginn: Start-Token oder letzter Encoder-Output)



Encoder-Decoder Visualisierung

- Sehr gutes Tutorial mit vielen Animationen: https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/
- Visualisierung: https://jalammar.github.io/images/seq2seq_6.mp4





Einschub: Suchstrategien während Inferenz

- **Greedy**: Wähle bei jedem Zeitschritt das Token mit der höchsten Wahrscheinlichkeit bis <EOS> vorhergesagt wird.
- Beispiel: 4 Token, 4 Zeitschritte
- Vorhersage:
  - A, B, C, <EOS>
  - $P = 0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$

Time	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

Einschub: Suchstrategien während Inferenz

- Eigenschaften von Greedy Search:
  - Schnell
  - Einfach zu implementieren
  - Optimiert nur Wahrscheinlichkeit für nächsten Token und nicht für die Gesamtsequenz

Time	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

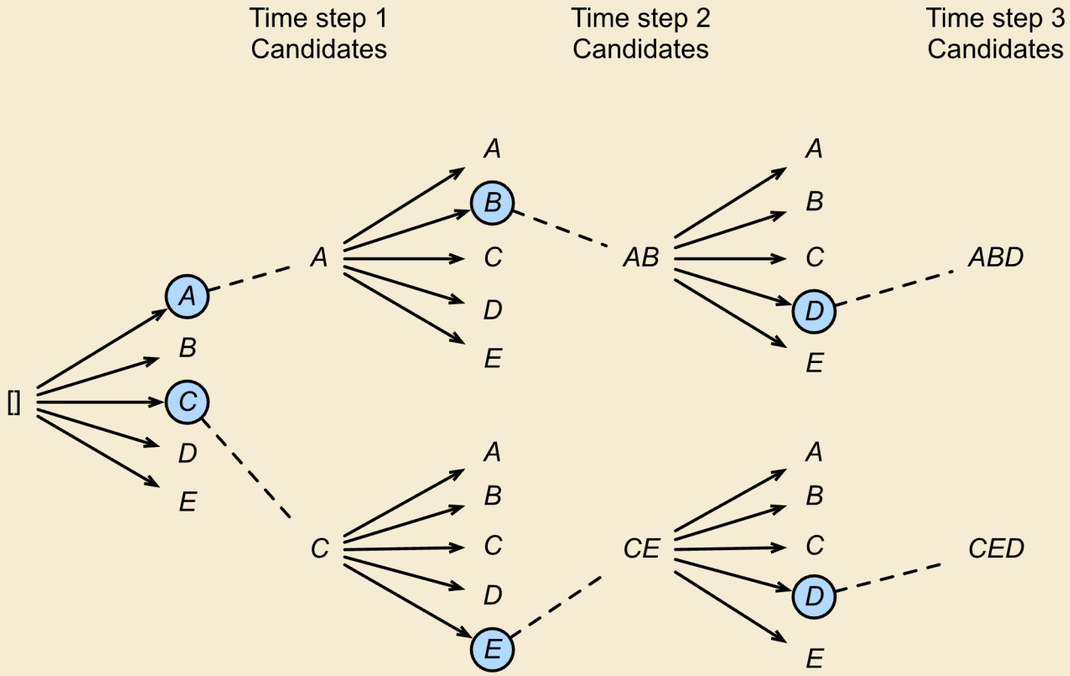
$P = 0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$

Time	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

$P = 0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$

Einschub: Suchstrategien während Inferenz

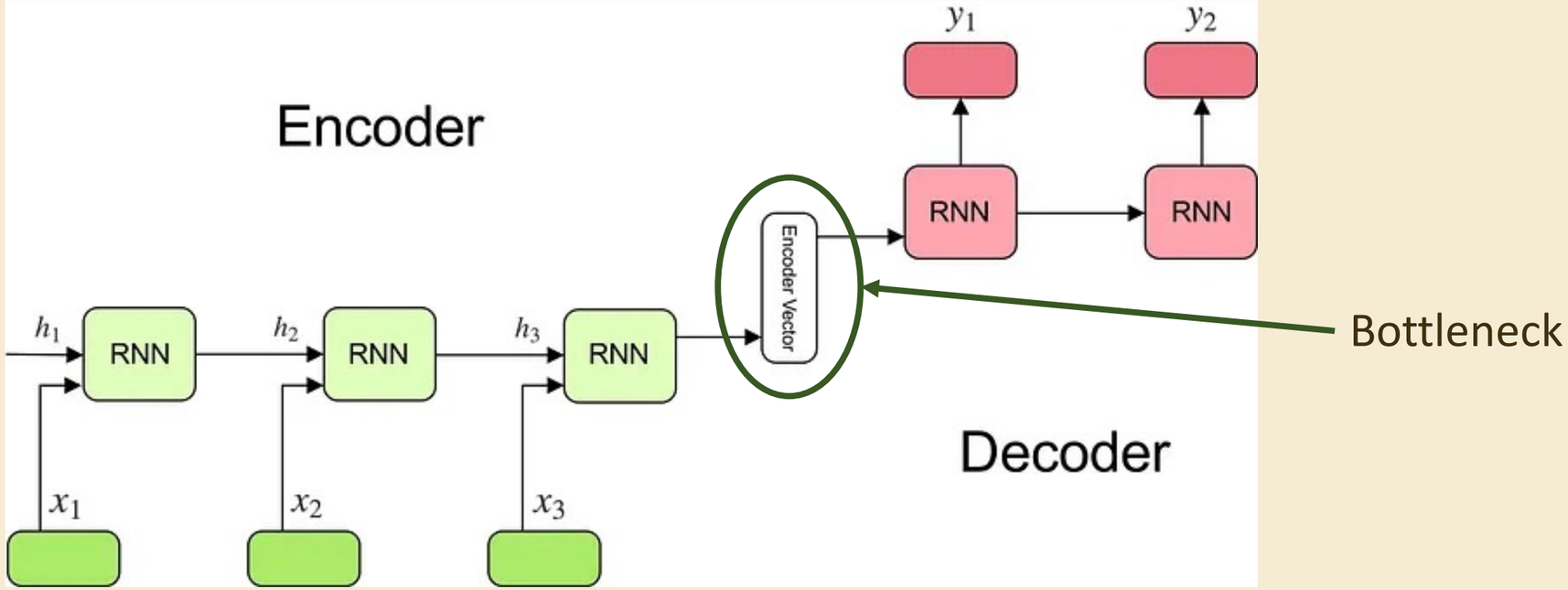
- Vollständige Suche (alle Kombinationen durchprobieren)
  - Zeitkomplexität: $O(V^n)$ (V: Vokabular, n: Sequenzlänge) → nicht praktikabel
- Beam Search



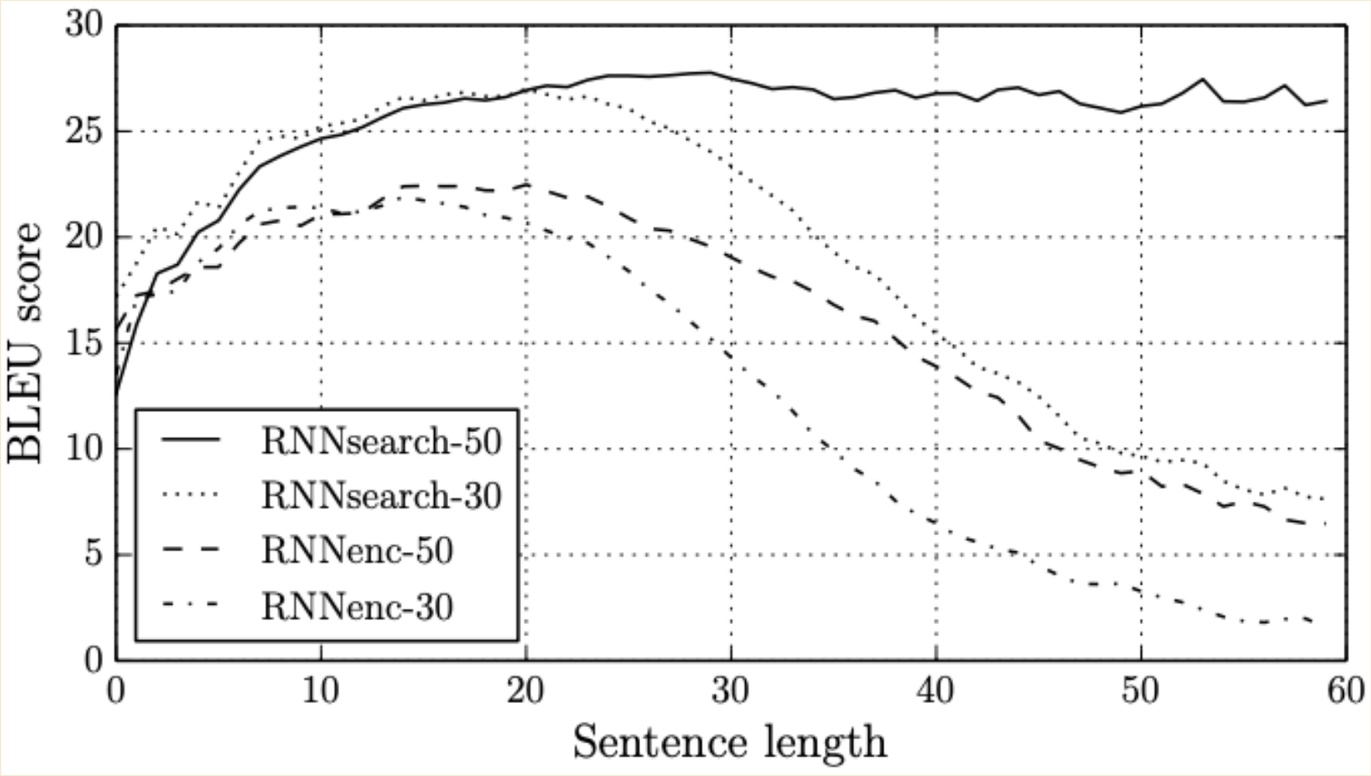
https://d2l.ai/chapter_recurrent-modern/beam-search.html

Problem der Encoder-Decoder Architektur

- **Bottleneck:** Gesamter Inhalt einer Sequenz (z.B. eines Satzes) wird **im finalen Tensor des Encoders** abgebildet
- Dieser hat eine feste Größe und muss Inhalte beliebiger Länge repräsentieren können



Problem der Encoder-Decoder Architektur



- RNNenc-*: Sequence2Sequence Modelle
- RNNsearch-*: Seq2Seq mit Attention (sie folgende Folien)

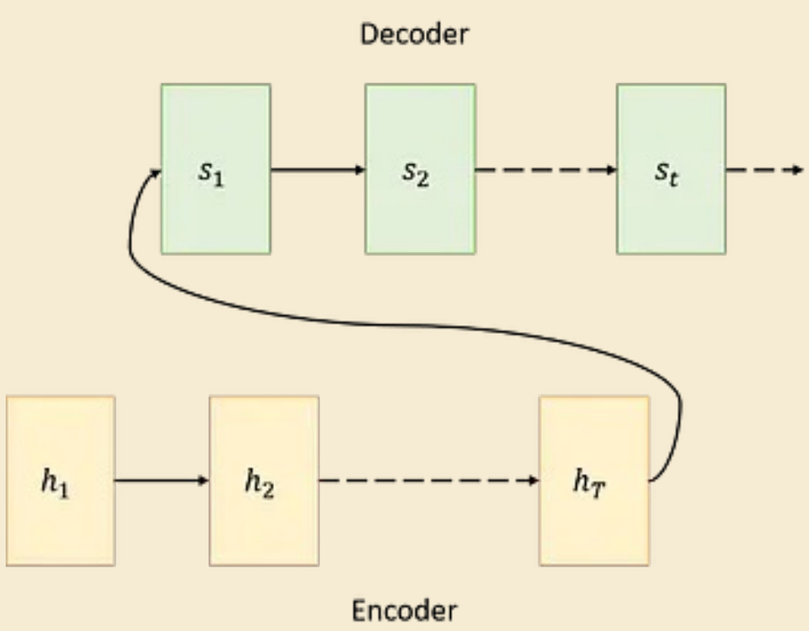
Bahdanau, Dzmitry & Cho, Kyunghyun & Yere, Yere. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. https://arxiv.org/abs/1409.0473

Attention – Idee

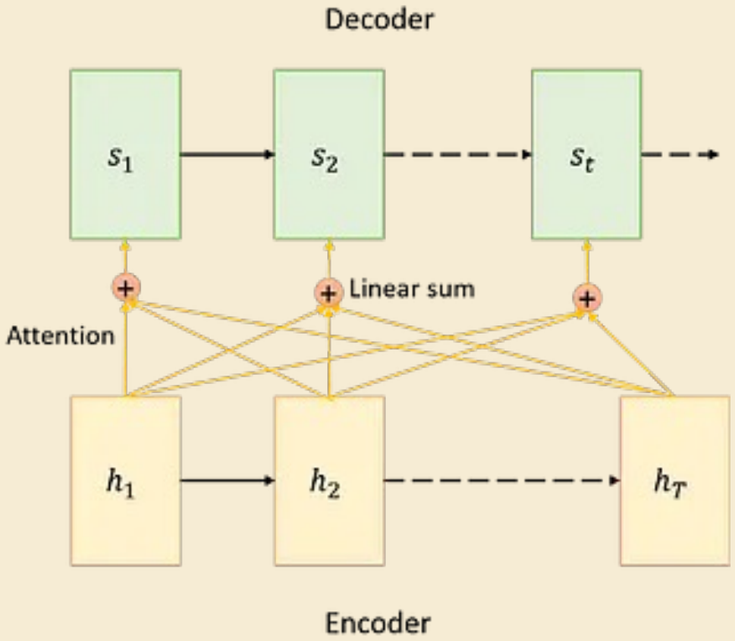
- Decoder wird so erweitert, so dass er zu jedem Zeitschritt den **gesamten Input** sieht und die **Relevanz** für den nächsten Zeitschritt bewertet.
- **Umsetzung:** Anstelle nur des letzten „hidden state“ des Encoders bekommt der Decoder **Zugriff auf alle „hidden states“** des Encoders
  - Zu jedem Zeitschritt entscheidet der Decoder, wie relevant ein bestimmter „hidden state“ ist.
- ”Bahdanau”-Attention

Bahdanau, Dzmitry & Cho, Kyunghyun & Yere, Yere. (2014). Neural Machine Translation by Jointly Learning to Align and Translate.
https://arxiv.org/abs/1409.0473

Attention Umsetzung (1)



(a) Vanilla Encoder Decoder Architecture

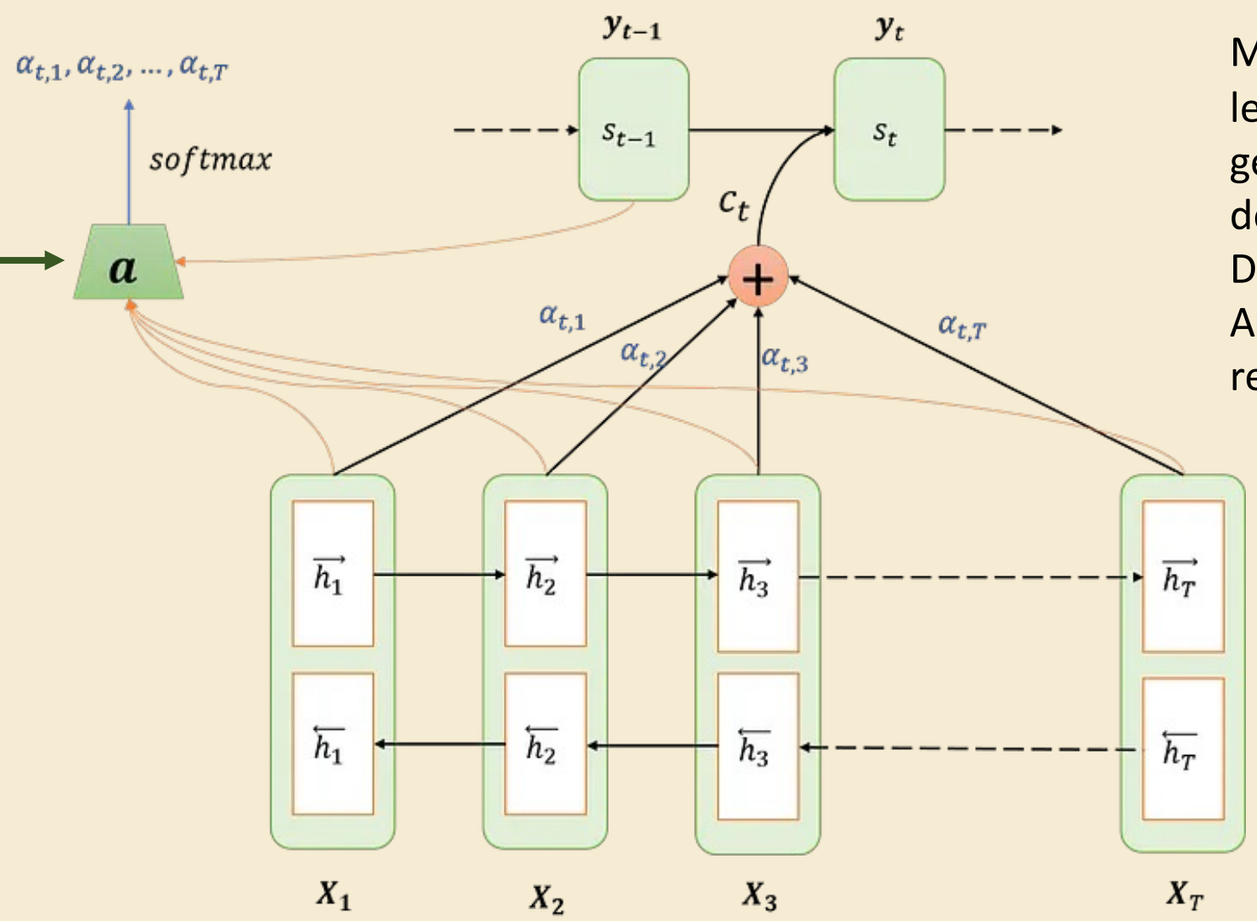


(b) Attention Mechanism

https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b

Attention Umsetzung (2)

Neuronales Netzwerk (d.h. Gewichte α werden gelernt)



Mit Hilfe der Gewichte $\alpha_{t,i}$ kann der Decoder lernen, welcher Teil des Inputs zu einem gegebenen Zeitpunkt, also für die Vorhersage des nächsten Tokens, am wichtigsten ist.
Der Decoder kann sozusagen seine Aufmerksamkeit (”Attention”) auf die relevanten Teile des Inputs lernen.

https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b

(Bahdanau-) Attention: Original-Formulierung

3.1 Decoder: General Description

In a new model architecture, we define each conditional probability in Eq. (2) as:

$$p(y_i|y_1,\ldots,y_{i-1},\mathbf{x}) = g(y_{i-1}, s_i, c_i), \tag{4}$$

where s_i is an RNN hidden state for time i, computed by

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

It should be noted that unlike the existing encoder–decoder approach (see Eq. (2)), here the probability is conditioned on a distinct context vector c_i for each target word y_i.

The context vector c_i depends on a sequence of *annotations* $(h_1, \cdots, h_{T_x})$ to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i-th word of the input sequence. We explain in detail how the annotations are computed in the next section.

The context vector c_i is, then, computed as a weighted sum of these annotations h_i:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j. \tag{5}$$

The weight α_{ij} of each annotation h_j is computed by

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \tag{6}$$

where

$$e_{ij} = a(s_{i-1}, h_j)$$

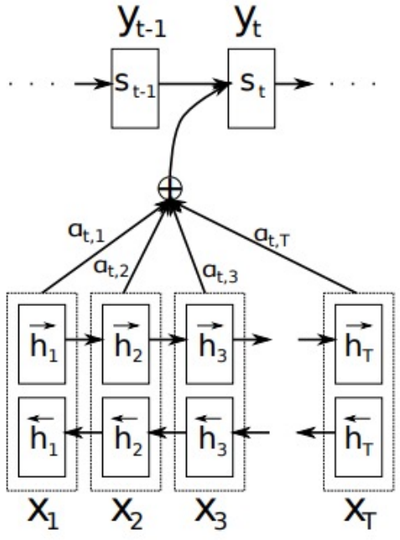
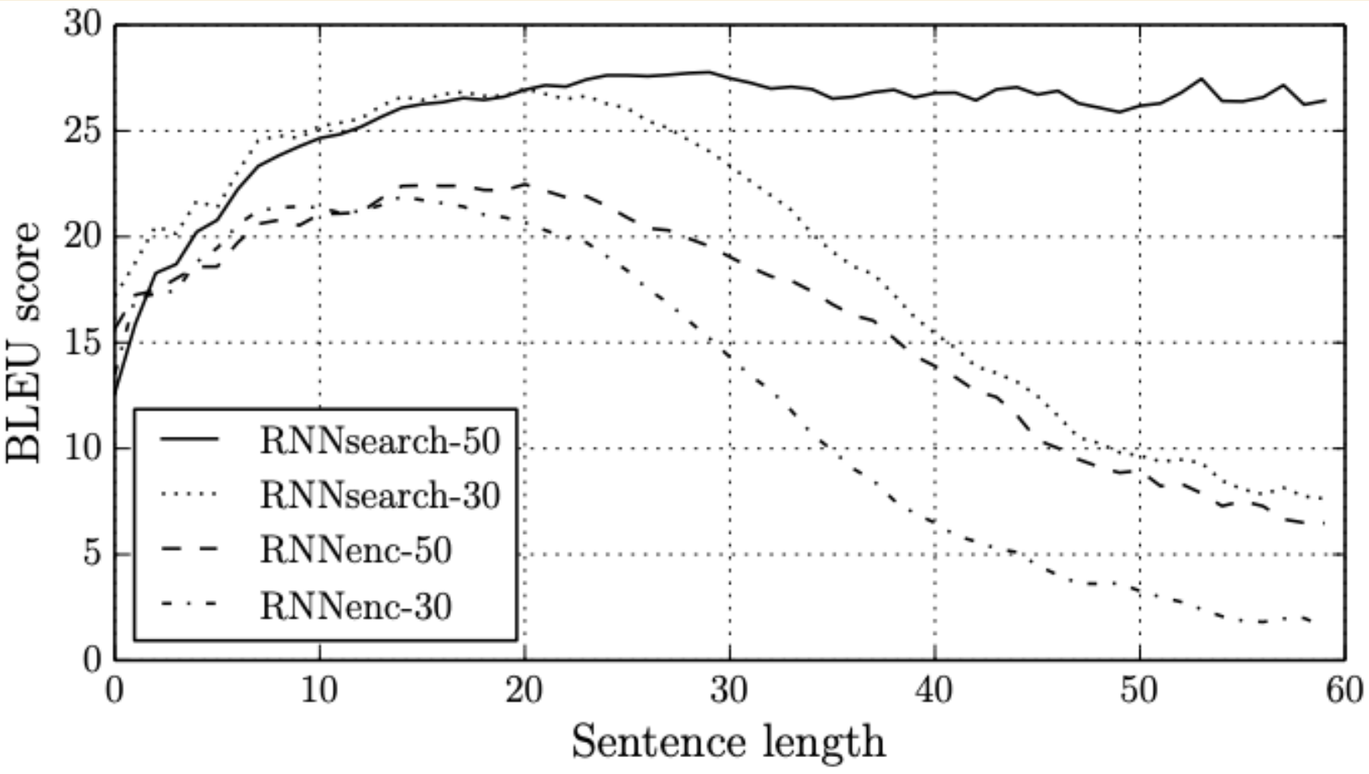


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.

Bahdanau, Dzmitry & Cho, Kyunghyun & Yere, Yere. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. https://arxiv.org/abs/1409.0473

"… is an alignment model which scores how well the inputs around position j and the output at position i match. The score is based on the RNN hidden state s_{i-1} (just before emitting y_i , Eq. (4)) and the j-th annotation h_j of the input sentence. **We parametrize the alignment model a as a feedforward neural network which is jointly trained with all the other components of the proposed system.**"

Encoder-Decoder Architektur mit und ohne Attention

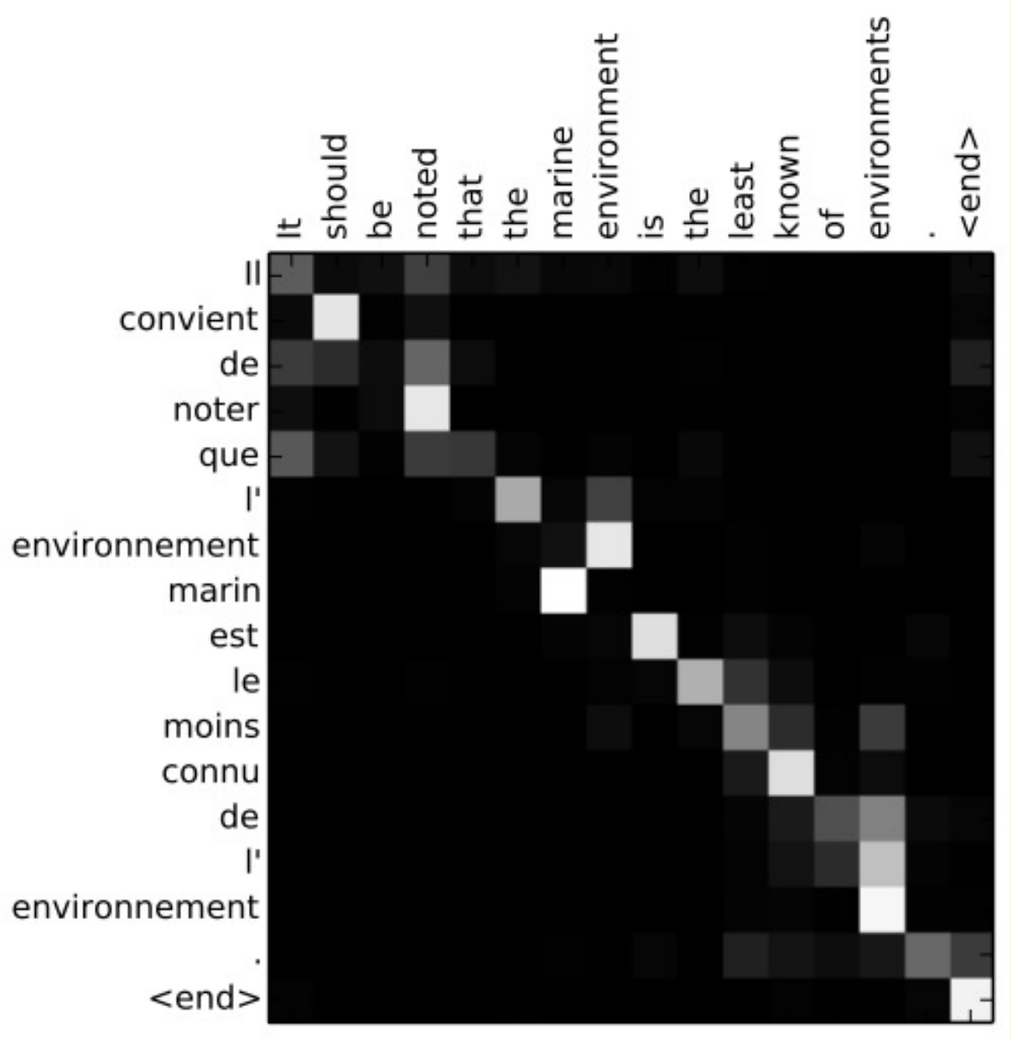


Attention Mechanismus verbessert die Leistung bei Langen Sequenzen deutlich.

- RNNenc-*: Sequence2Sequence Modelle
- RNNsearch-*: Seq2Seq mit Attention

Bahdanau, Dzmitry & Cho, Kyunghyun & Yere, Yere. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. https://arxiv.org/abs/1409.0473

Attention – Visualisierung



- Bild zeigt die Attention-Gewichte
- "The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight α_{ij} of the annotation of the j-th source word for the i-th target word (see Eq. (6)), in grayscale (0: black, 1: white)."

Bahdanau, Dzmitry & Cho, Kyunghyun & Yere, Yere. (2014). Neural Machine Translation by Jointly Learning to Align and Translate.
https://arxiv.org/abs/1409.0473